

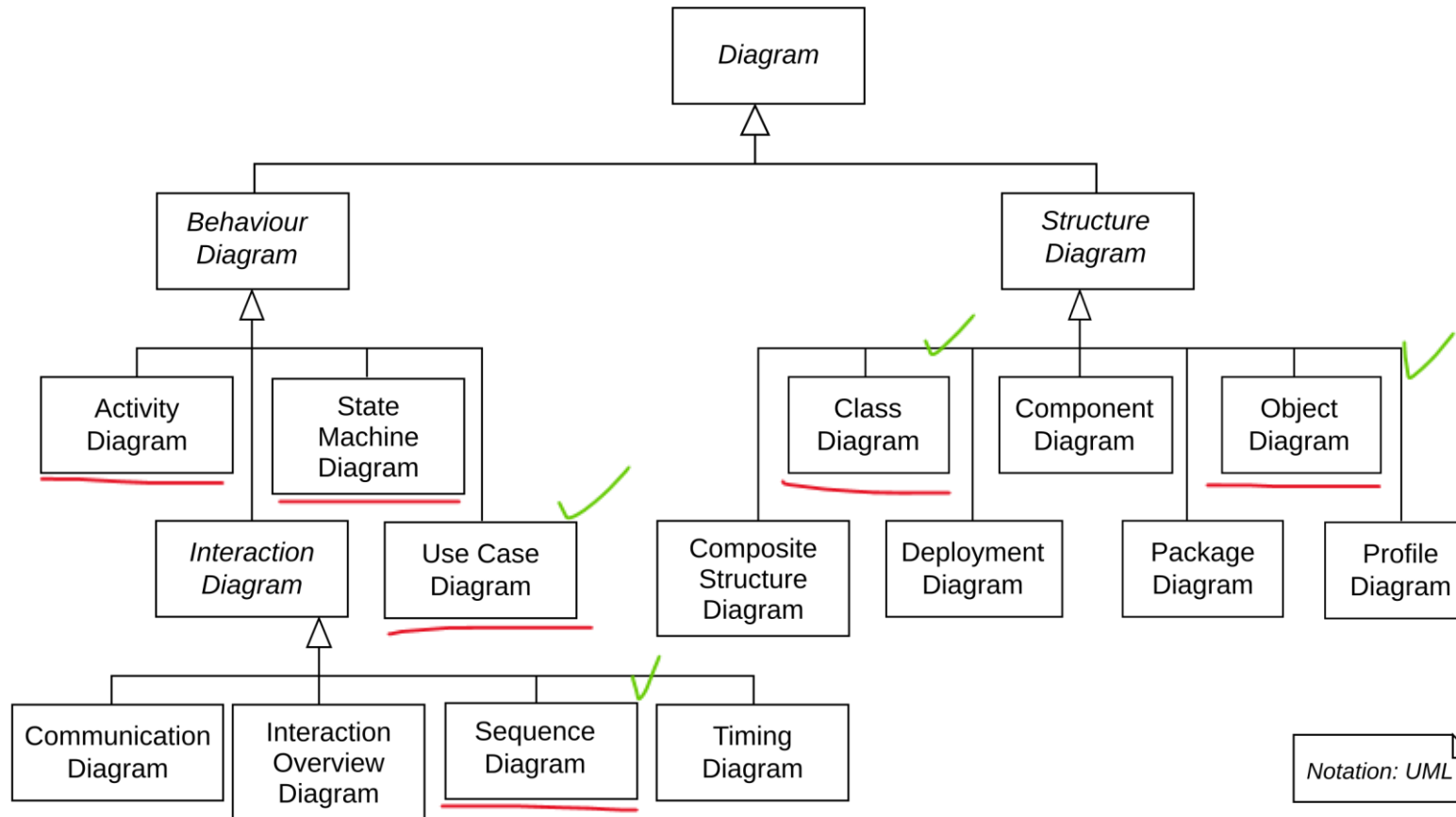
UML Diagramme



Einführung in UML (Unified Modeling Language)

- **Was ist UML?**
 - Eine standardisierte Sprache zur Modellierung von Softwaresystemen.
 - Dient der Visualisierung, Spezifikation, Konstruktion und Dokumentation.
- **Wichtig:** UML ist **keine Programmiersprache**, sondern eine **Modellierungssprache**.
- **Die zwei Hauptkategorien:**
 - **Strukturdiagramme:** Zeigen den statischen Aufbau (z. B. Klassendiagramm).
 - **Verhaltensdiagramme:** Zeigen das dynamische Verhalten und Abläufe (z. B. Aktivitäts-, Sequenz-, Zustandsdiagramm).

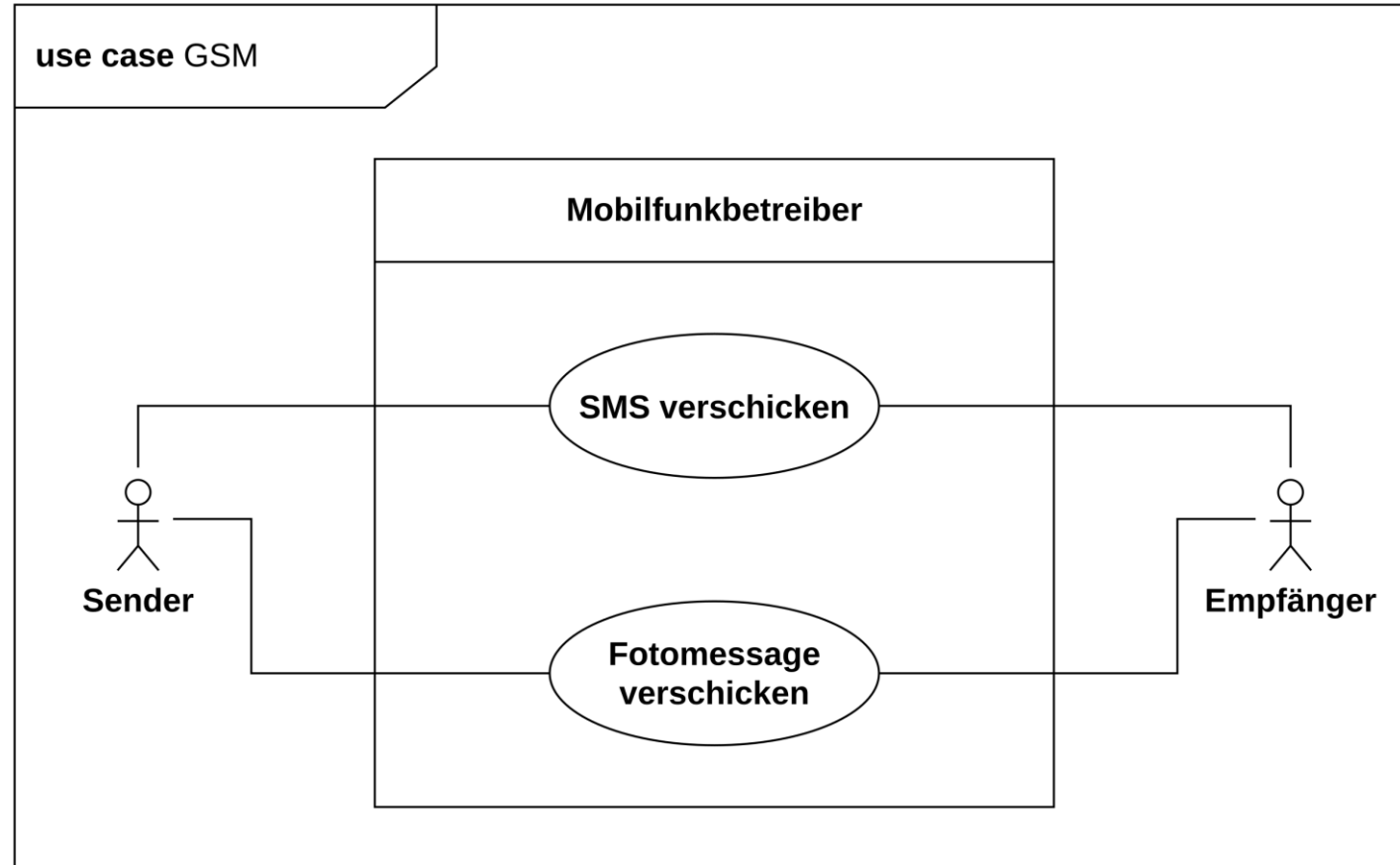
Übersicht UML Diagramm Hierarchie



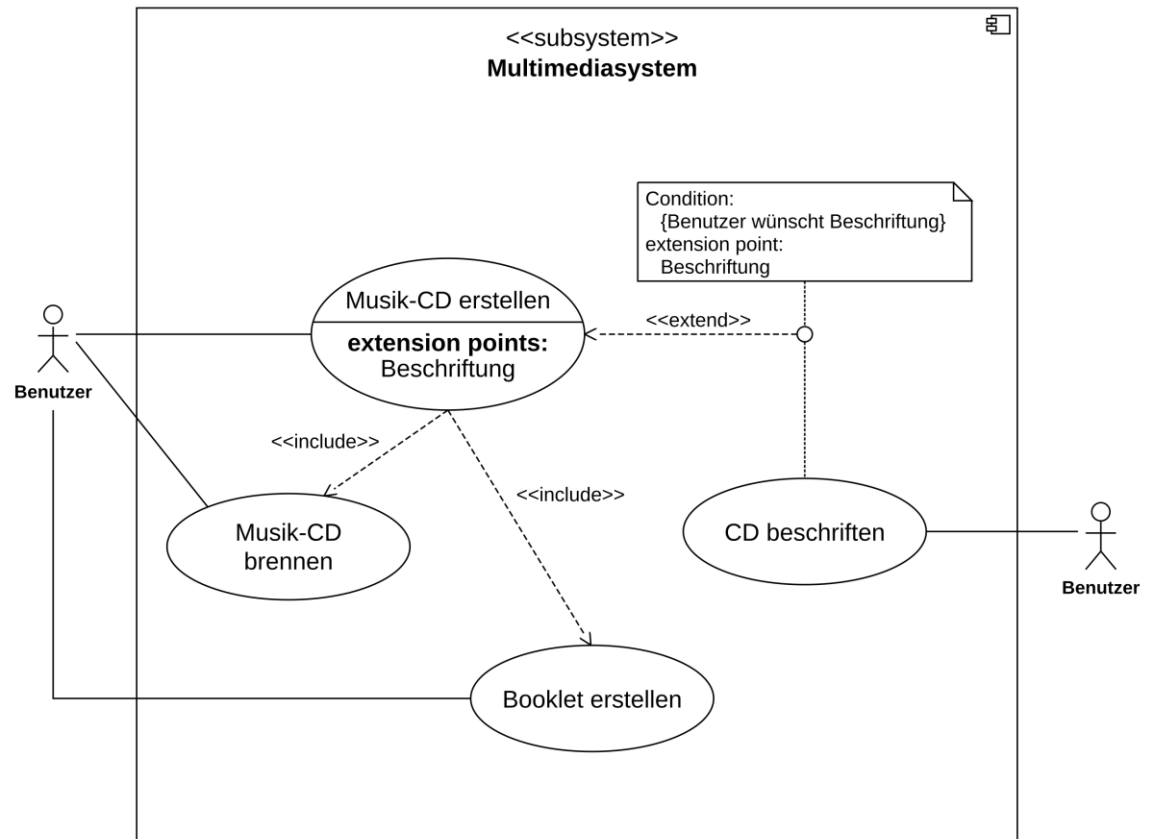
Use Case Diagramm (Anwendungsfalldiagramm)

- **Zweck:**
Beschreibt, **wer** (Akteur) **was** (Anwendungsfall) mit dem System macht. Es zeigt die Anforderungen aus Nutzersicht.
 - **Akteur (Männchen):** Benutzer oder externes System.
 - **Use Case (Ellipse):** Eine Funktion des Systems.
 - **Systemgrenze (Rechteck):** Trennt System von der Außenwelt.
 - **Beispiel-Szenario:** GSM Mobilfunk System.

Use Case Anwendungsfalldiagramm

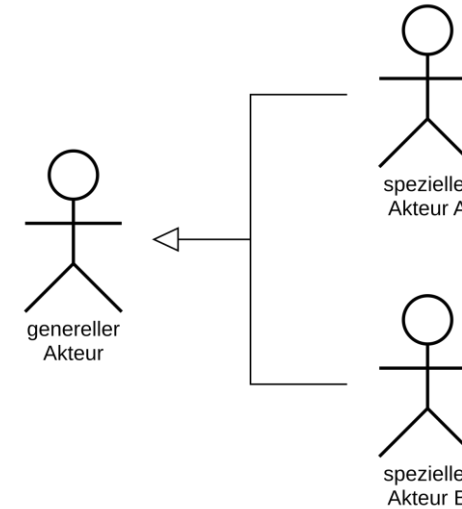


Use Case Diagramm komplexeres Beispiel

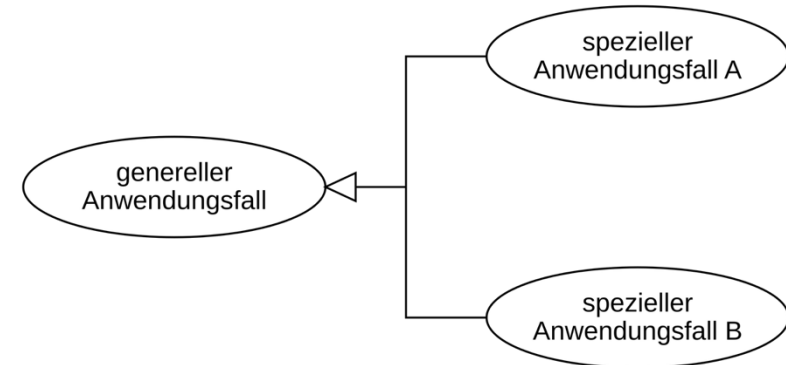


Generalisierung Use Case Diagramm

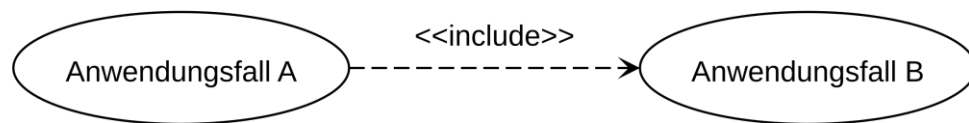
- Generalisierung von Akteuren



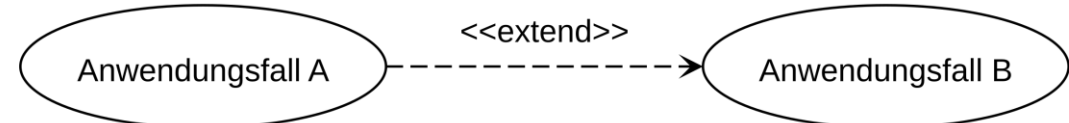
- Generalisierung von Anwendungsfällen



Use Case Diagramm Beziehungen

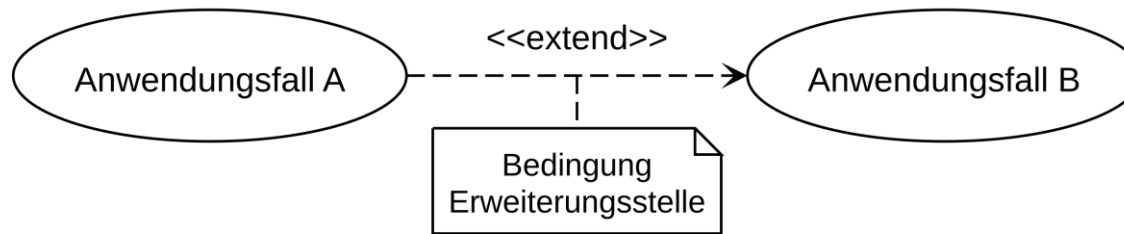


Include-Beziehung im Anwendungsfalldiagramm, wobei Anwendungsfall A den Anwendungsfall B beinhaltet.



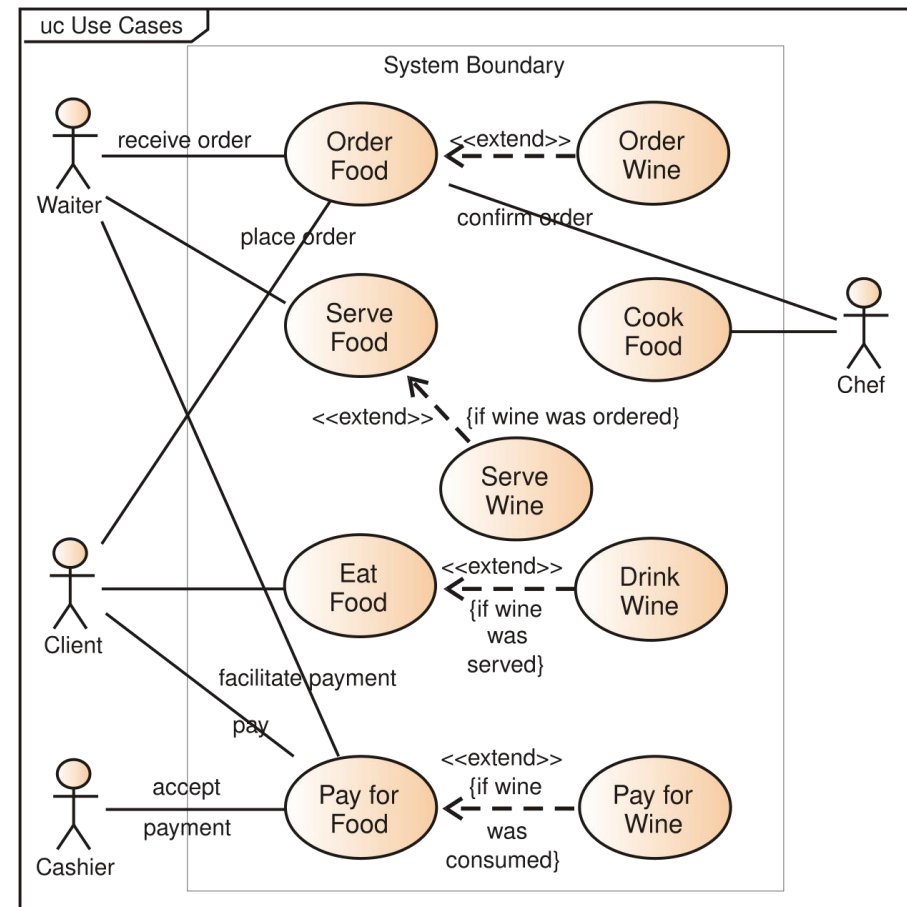
Extend-Beziehung im Anwendungsfalldiagramm, wobei Anwendungsfall A den Anwendungsfall B erweitert.

Use Case Diagramme – extension points



Extend-Beziehung mit extension point, wobei Anwendungsfall A den Anwendungsfall B unter der angegebenen Bedingung erweitert.

Beispiel Use Case Diagramm



Klassendiagramm (Struktur)

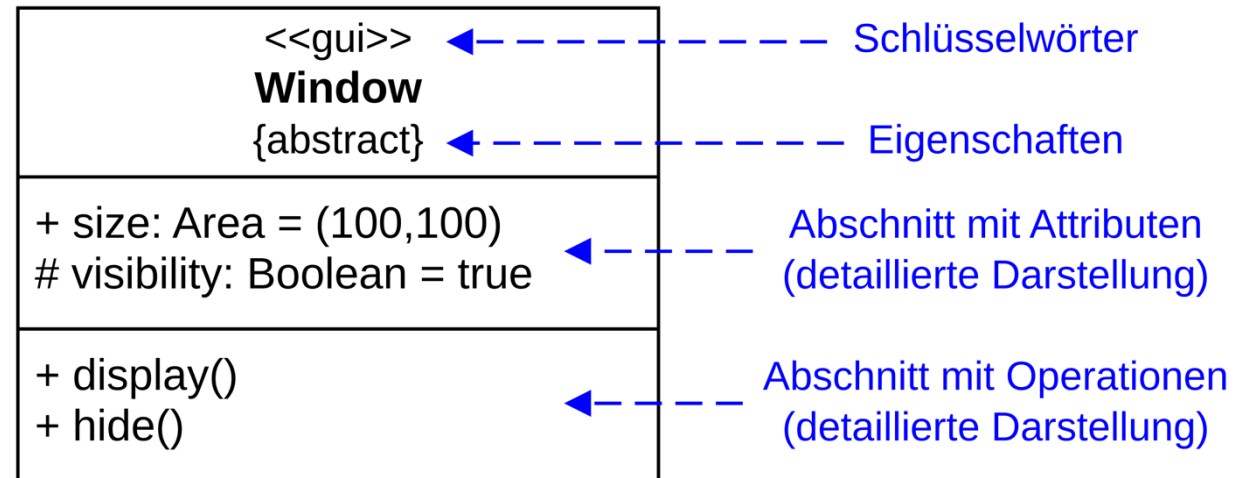
- **Klassendiagramm (Struktur)**

- **Zweck:**

Das wichtigste Diagramm für OOP. Zeigt Klassen, Attribute, Methoden und Beziehungen (Vererbung, Assoziation).

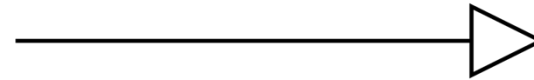
- **Syntax:**

- + Public
- - Private
- # Protected



Klassendiagramme Syntax

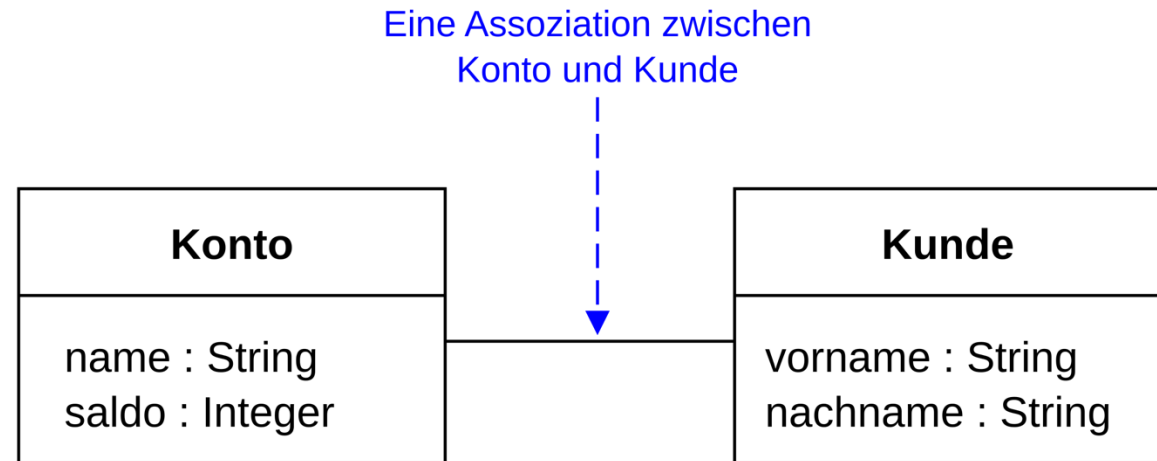
- **Generalisierung (in Pfeilrichtung)**



- **Assoziation**

Eine **Assoziation** beschreibt eine **strukturelle Beziehung** zwischen **Klassen**.

An den Enden von Assoziationen sind häufig Multiplizitäten vermerkt.



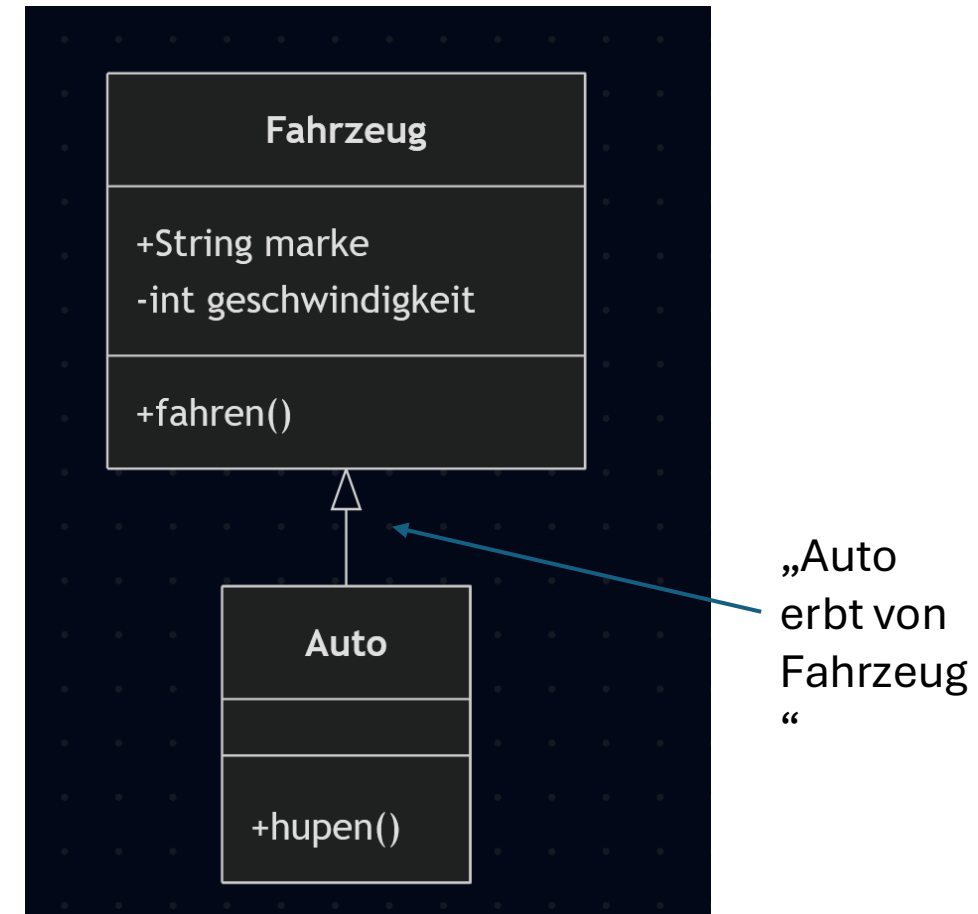
Klassendiagramm

```
class Fahrzeug:
    def __init__(self, marke: str):
        self.marke = marke
        self._geschwindigkeit = 0

    def fahren(self):
        print(f"{self.marke} fährt.")

class Auto(Fahrzeug): # Vererbung
    def hupen(self):
        print("Tuut tuut")

# Nutzung
mein_auto = Auto("BMW")
mein_auto.fahren()
```

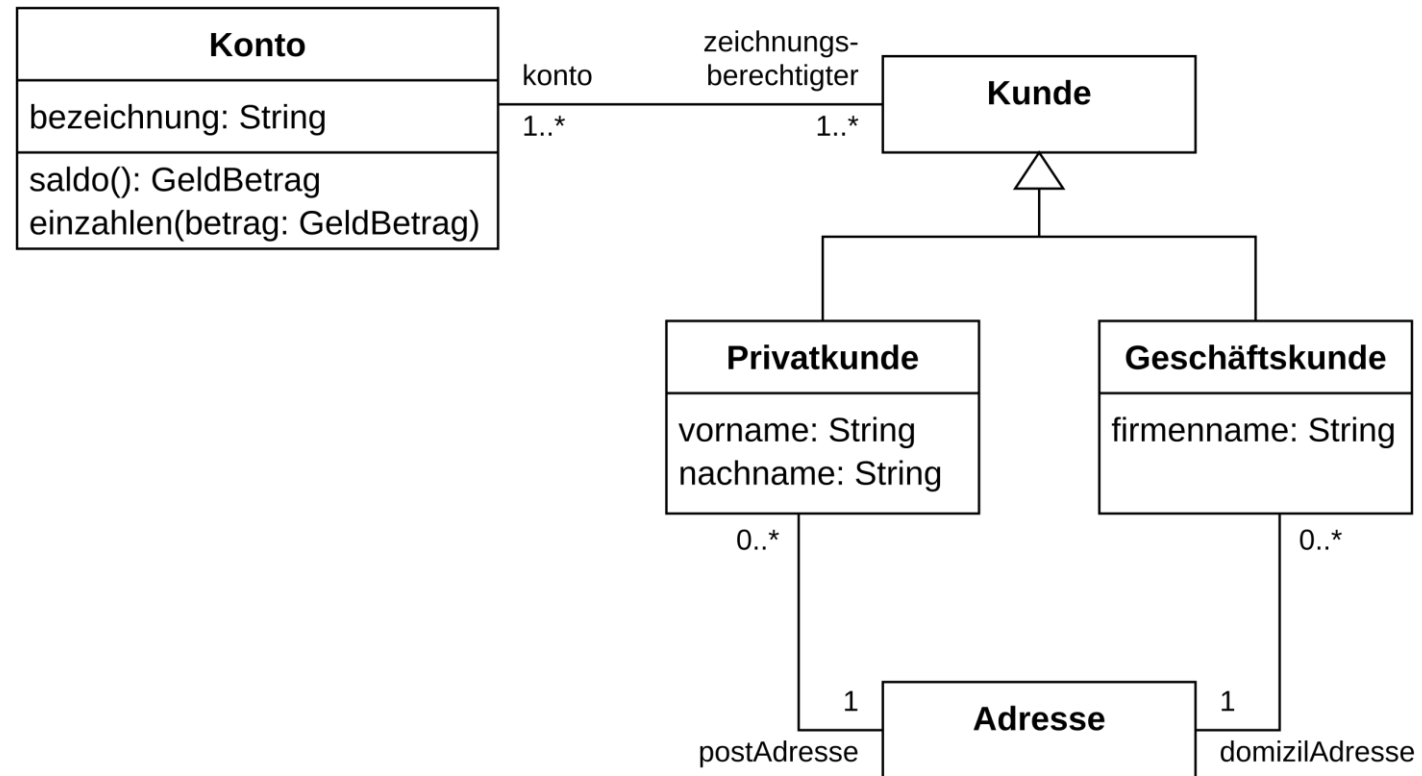


Multiplizität/Kardinalität (UML)

- Sie gibt an, **wie viele Objekte** beteiligt sind.
- wird oft mit **untereSchranke..obereSchranke** angegeben,
- * bei der oberen Schranke für den Wert ***unbeschränkt*** steht
- * oftmals als Abkürzung für 0..* verwendet wird

Notation	Bedeutung
1	genau eins
0..1	null oder eins
* oder 0..*	beliebig viele
1..*	mindestens eins
2..5	zwischen 2 und 5

Klassendiagramm

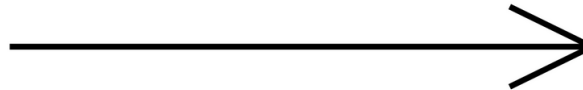


Klassendiagramm

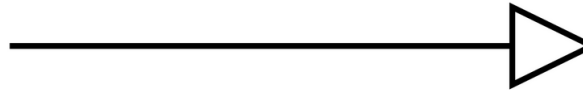
Assoziation



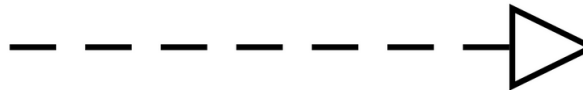
Gerichtete Assoziation



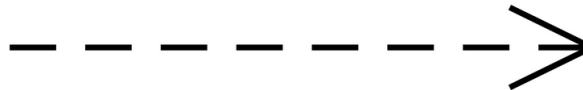
Vererbung



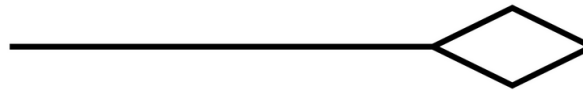
Realisierung/Implementation



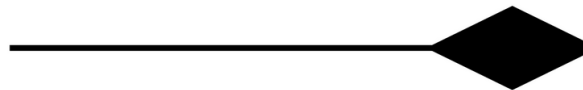
Abhängigkeit



Aggregation



Komposition



https://en.wikipedia.org/wiki/Class_diagram#/media/File:Uml_classes_en.svg

Interfaces

- **Was ist ein Interface in Java?**
- Ein Interface kann man sich wie einen **Vertrag** vorstellen. Es legt fest, *was* eine Klasse tun muss, aber nicht, *wie* sie es tut.
- **Hauptmerkmale:**
 - **Methodensignaturen:** Ein Interface enthält normalerweise nur Methodenköpfe (Name, Rückgabetyp, Parameter), aber keinen Code im Rumpf.
 - **Keine Instanziierung:** Man kann kein Objekt von einem Interface mit `new` erstellen.
 - **Mehrfachvererbung:** Eine Klasse kann nur von *einer* Superklasse erben (`extends`), aber *beliebig viele* Interfaces implementieren (`implements`).
 - **Abstraktion:** Es trennt die Definition einer Schnittstelle von ihrer konkreten Implementierung.

Interfaces in Java

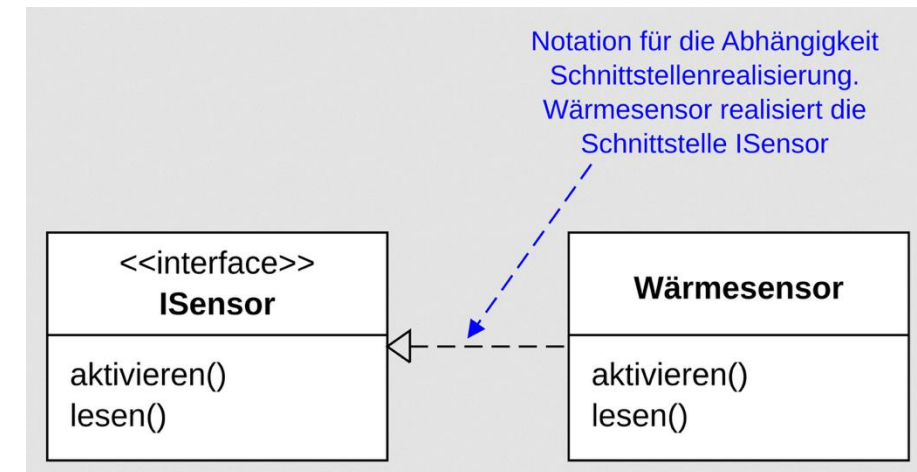
```
// Definition des Interface
public interface Tier {
    void macheGeräusch(); // implizit public und abstract
    void bewegeSich();
}

// Eine Klasse "implementiert" das Interface
public class Hund implements Tier {
    @Override
    public void macheGeräusch() {
        System.out.println("Wuff Wuff!");
    }

    @Override
    public void bewegeSich() {
        System.out.println("Der Hund rennt.");
    }
}
```

Darstellung in UML

- In der UML (Unified Modeling Language) wird ein Interface ähnlich wie eine Klasse dargestellt, aber mit spezifischen Kennzeichnungen.
- **A. Das Interface-Symbol**
 - Ein Interface wird als Rechteck dargestellt, genau wie eine Klasse. Um es zu unterscheiden, nutzt man ein **Stereotyp**:
 - Oben im Kasten steht in Guilloche-Klammern: **«interface»**.



https://upload.wikimedia.org/wikipedia/commons/thumb/e/e1/UmlCd_Interface-2.svg/2560px-UmlCd_Interface-2.svg.png CC BY-SA 3.0 Gubaer

Klassendiagramme - Gerichtete Assoziation (Directed Association)

Erklärung:

Eine gerichtete Assoziation beschreibt eine dauerhafte Beziehung zwischen zwei Klassen. Das wichtigste Merkmal ist die **Navigierbarkeit**: Klasse A kennt Klasse B und hat oft ein Attribut von diesem Typ, aber Klasse B kennt Klasse A nicht.

Symbol: Eine **durchgezogene** Linie mit einer **offenen** Pfeilspitze.

Beispiel: Ein Benutzer hat ein Profil. Der Benutzer greift auf sein Profil zu, aber das Profil-Objekt muss nicht wissen, welcher Benutzer es besitzt.



Klassendiagramme Abhängigkeit (Dependency)

Erklärung:

Eine **Abhängigkeit** ist eine schwächere, meist kurzfristige Beziehung. Man nennt sie oft eine „Benutzt-Beziehung“. Klasse A benötigt Klasse B nur für eine bestimmte Aufgabe (z. B. als Parameter in einer Methode oder als lokale Variable), speichert sie aber nicht dauerhaft als Attribut. Wenn sich Klasse B ändert, kann das Auswirkungen auf Klasse A haben.

•**Symbol:** Eine **gestrichelte** Linie mit einer **offenen** Pfeilspitze.

•**Beispiel:** Ein RechnungsDrucker benötigt ein Dokument-Objekt, um die Methode drucken() auszuführen. Er "besitzt" das Dokument nicht dauerhaft.



Beispiel kombiniert

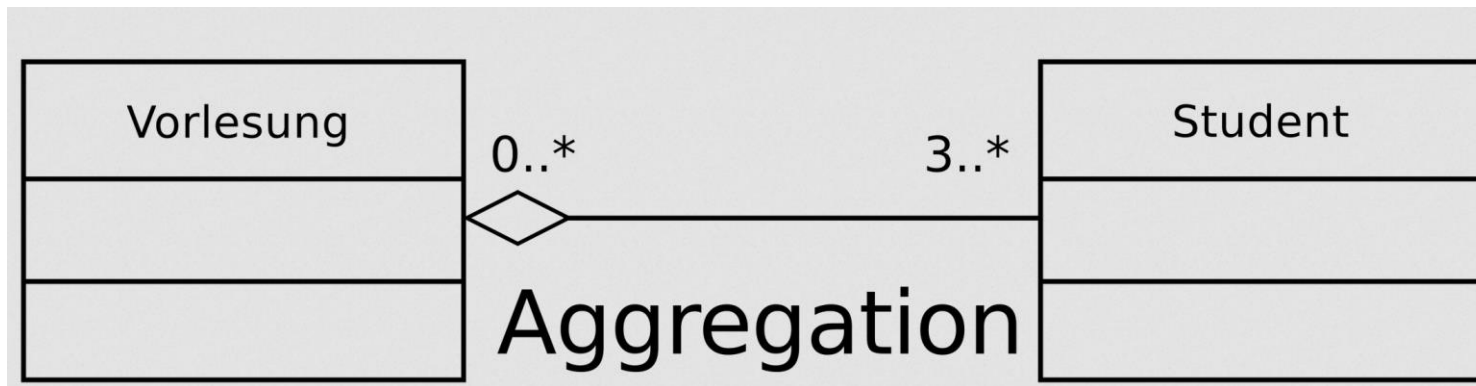
1. Der Kunde hat eine **gerichtete Assoziation** zum Warenkorb (er besitzt ihn über den gesamten Zeitraum).

2. Der Kunde hat eine **Abhängigkeit** zum Bezahl dienst (er benötigt ihn nur in dem Moment, in dem die Methode bestellen aufgerufen wird).



Aggregation und Komposition

- **Aggregation (schwache Teil-Ganzes-Beziehung)**
 - Ein Objekt ist **Teil eines Ganzen** („teil von Beziehung“)
 - Der **Teil kann aber unabhängig existieren!** (z.B. Student)
 - In der grafischen Darstellung einer Aggregation kennzeichnet eine **nicht ausgefüllte Raute** das Ende, das mit dem Ganzen verbunden ist

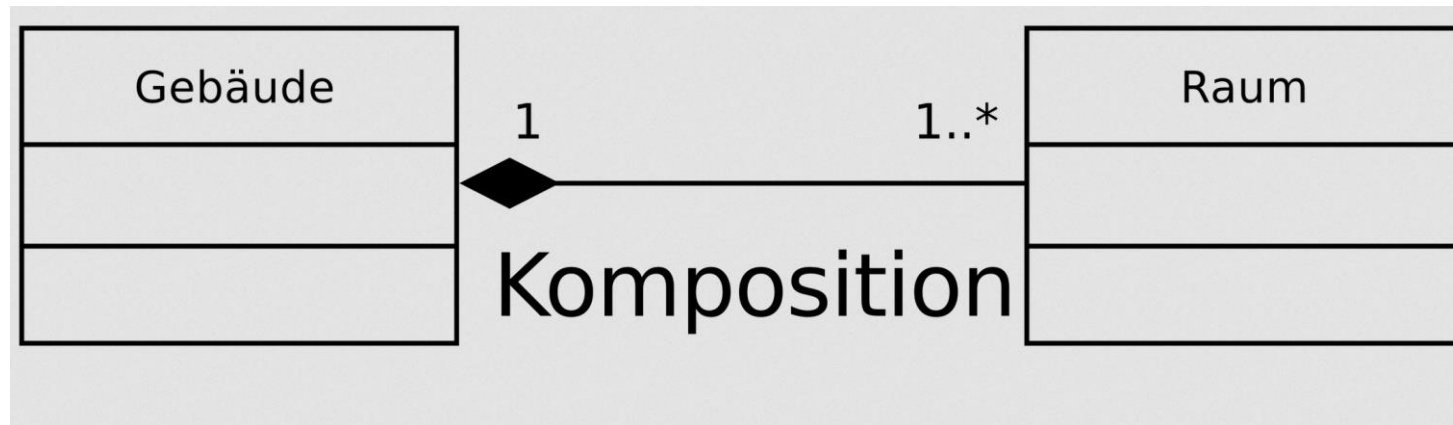


Beispiel Aggregation

- Team $\diamond$ — Spieler
→ Spieler können auch ohne Team existieren
- Bibliothek $\diamond$ — Buch
→ Buch existiert auch ohne Bibliothek

Aggregation und Komposition

- **Komposition (starke Teil-Ganzes-Beziehung)**
 - Spezialfall der Aggregation
 - Teil ist fester Bestandteil des Ganzen
 - Teil existiert **nicht** ohne Ganzes!
 - **ausgefüllte Raute**, kennzeichnet das Ende, das mit dem Ganzen verbunden ist.

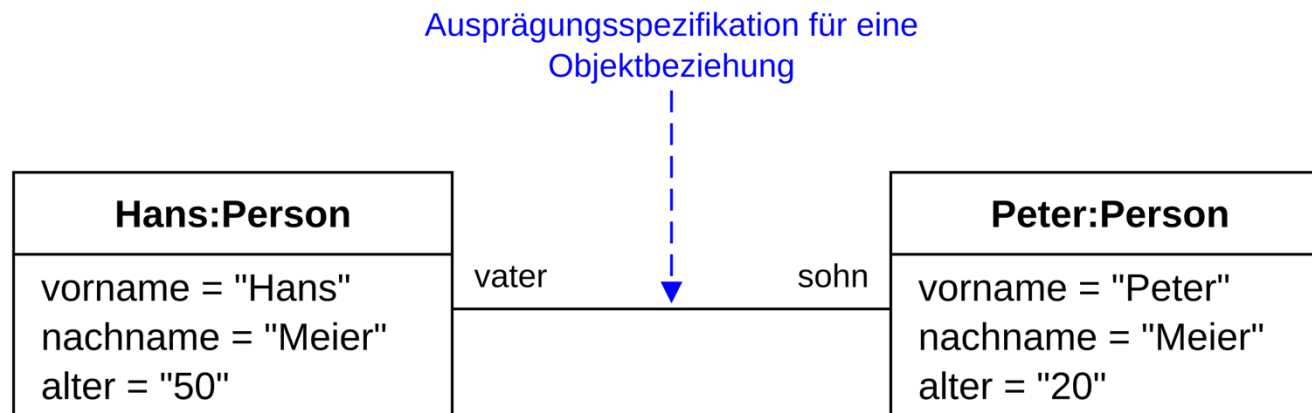


Beispiel Komposition

- **Beispiel**
- Haus ◆— Raum
 - Wird das Haus zerstört, verschwinden die Räume
- Bestellung ◆— Bestellposition
 - Position gehört genau zu einer Bestellung

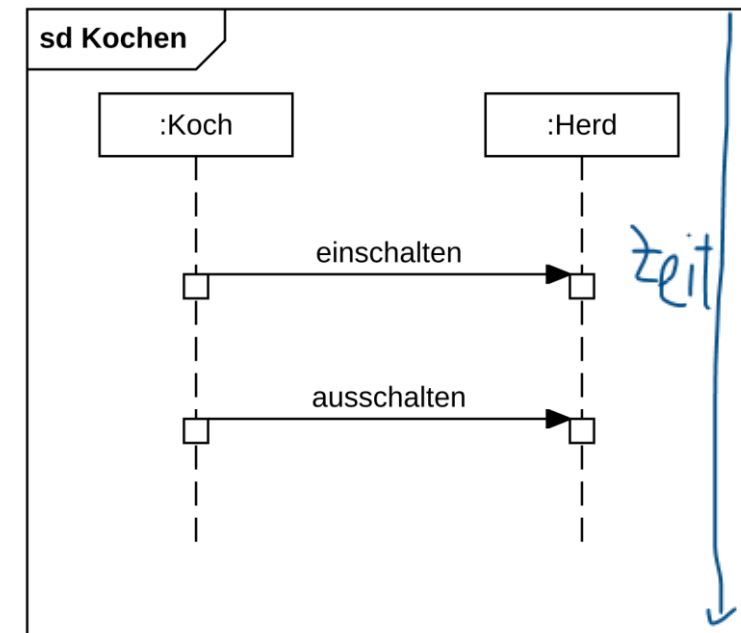
Objektdiagramm

- Das Objektdiagramm ist eine Spezifizierung des Klassendiagramms es stellt die Beziehung der tatsächlich erzeugten Diagramme zu einem bestimmten Zeitpunkt der Laufzeit dar.



Sequenzdiagramm (Verhalten / Interaktion)

- **Zweck:**
Zeigt den zeitlichen Ablauf von Nachrichten zwischen Objekten. Ideal, um zu verstehen, wie Objekte miteinander kommunizieren, um eine Aufgabe zu erfüllen.
- **Elemente:**
 - **Lebenslinie:** Gestrichelte vertikale Linie.
 - **Aktivierungsbalken:** Zeigt an, wann ein Objekt aktiv arbeitet.
 - **Nachrichten:** Pfeile (durchgezogen = Aufruf, gestrichelt = Antwort).

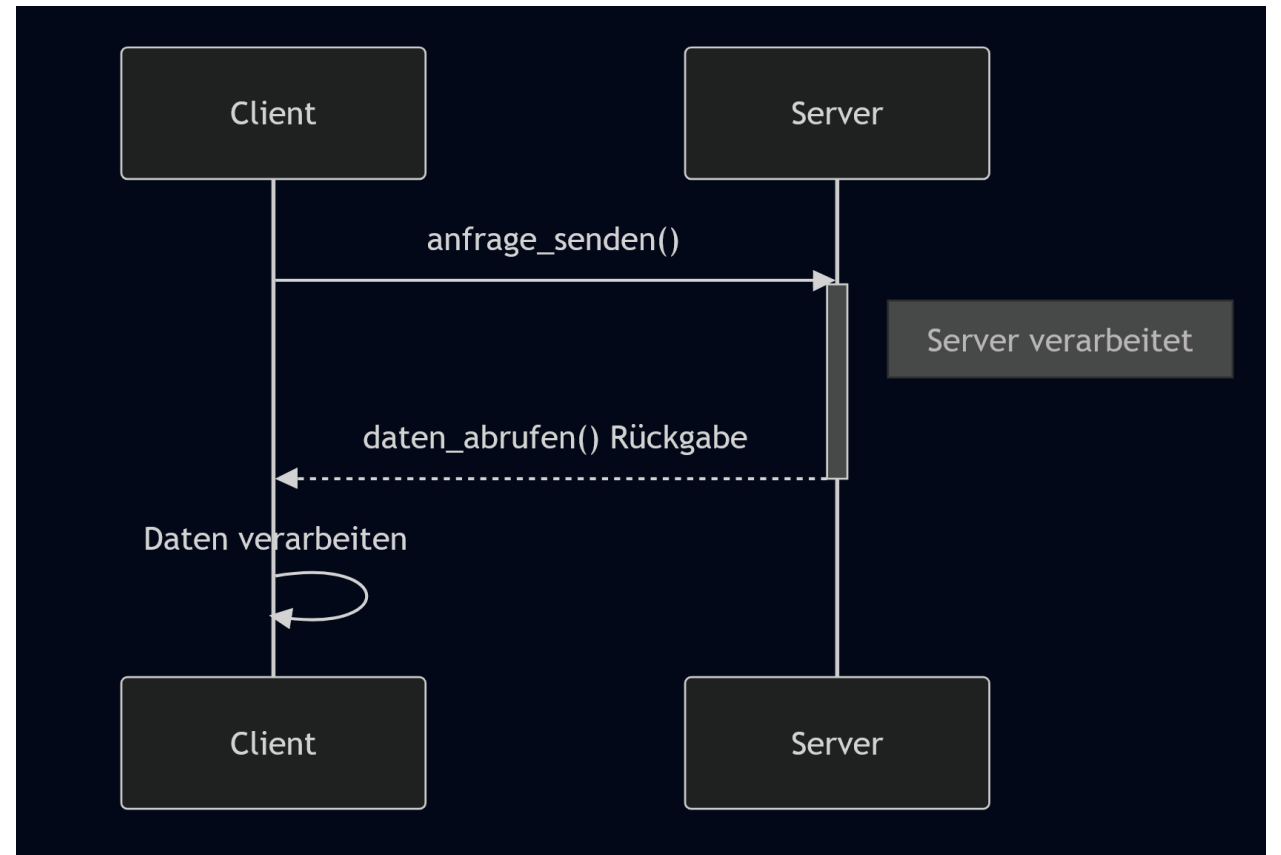


Sequenzdiagramm

```
class Server:
    def daten_abrufen(self):
        return "Datenpakete"

class Client:
    def anfrage_senden(self, server):
        print("Sende Anfrage...")
        antwort = server.daten_abrufen()
        print(f"Empfangen: {antwort}")

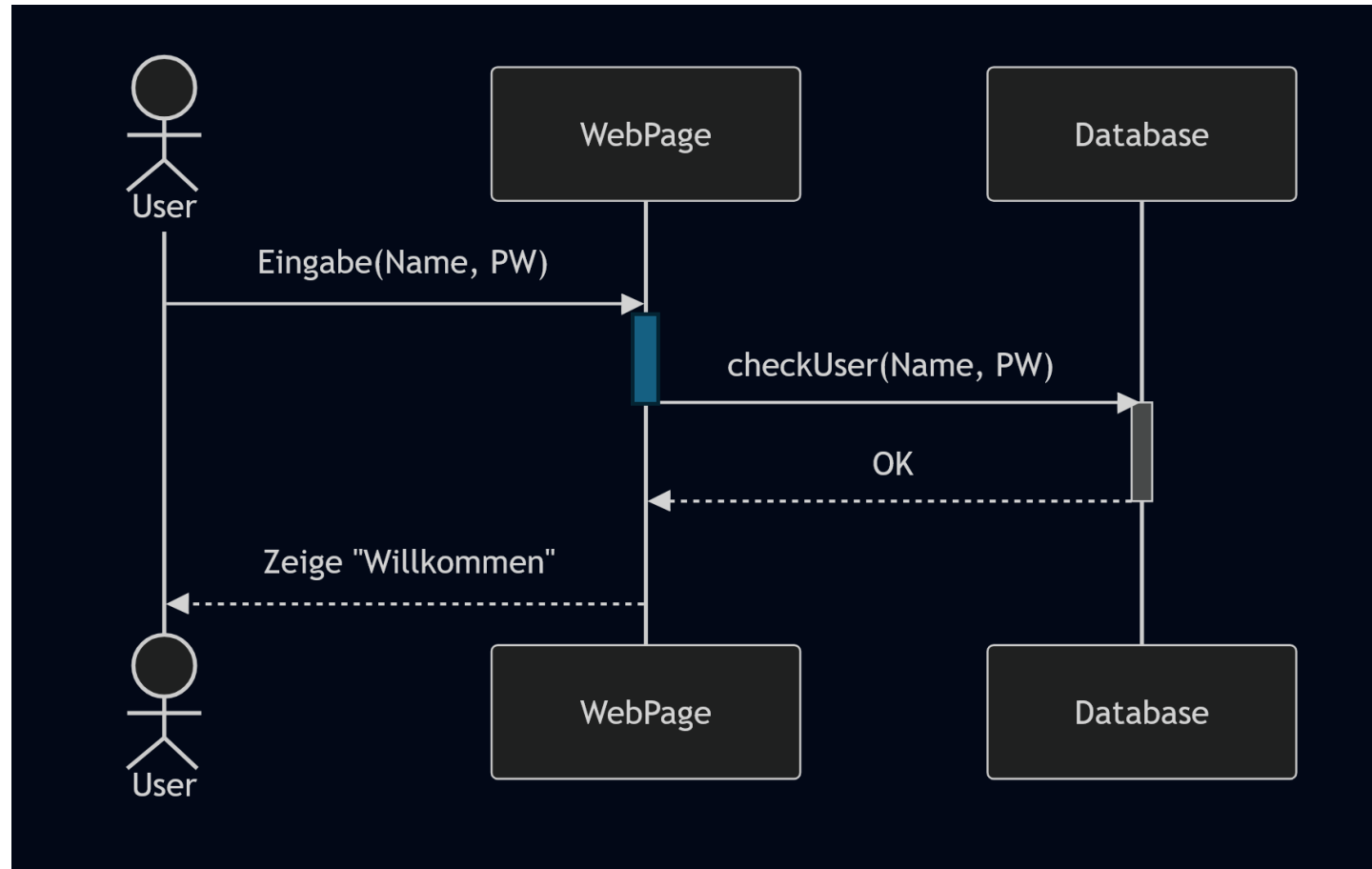
s = Server()
c = Client()
c.anfrage_senden(s)
```



Sequenzdiagramm Aufgabe

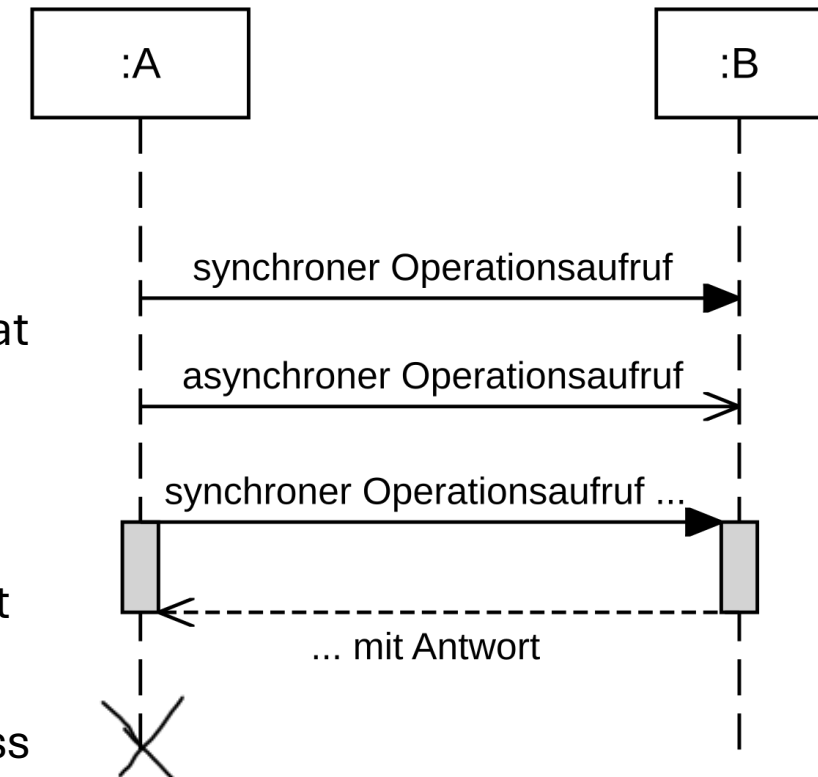
- **Szenario:** Ein Benutzer loggt sich auf einer Webseite ein.
Ablauf:
 - Benutzer gibt Daten ein.
 - WebPage sendet Daten an Database.
 - Database validiert und sendet "OK" zurück.
 - WebPage zeigt "Willkommen".
- Erstellen Sie ein Sequenzdiagramm des Ablaufs.

Lösung Sequenzdiagramm Aufgabe



Sequenzdiagramm (Nachrichten/Messages)

- Nachrichten (häufig Methodenaufrufe) werden als Pfeile zwischen den Lebenslinien dargestellt.
- Wir unterscheiden:
 - **Synchrone** Nachricht:
 - Symbol: durchgezogene Linie mit **gefüllter Pfeilspitze**
 - **Bedeutung:** Der Sender wartet, bis der Empfänger die Nachricht verarbeitet und eine Antwort zurückgesendet hat (blockierender Aufruf)
 - **Beispiel:** Ein Funktionsaufruf `getUser(id)`
 - **Asynchrone** Nachricht:
 - **Symbol:** Durchgezogene Linie mit **offener Pfeilspitze**.
 - **Bedeutung:** Der Sender sendet die Nachricht und arbeitet sofort weiter, ohne auf eine Antwort zu warten (nicht-blockierend).
 - **Beispiel:** Senden einer E-Mail in einem Hintergrundprozess oder Event-Trigger.



Sequenzdiagramm (Nachrichten/Messages)

- **Anwortnachricht (Return Message)**

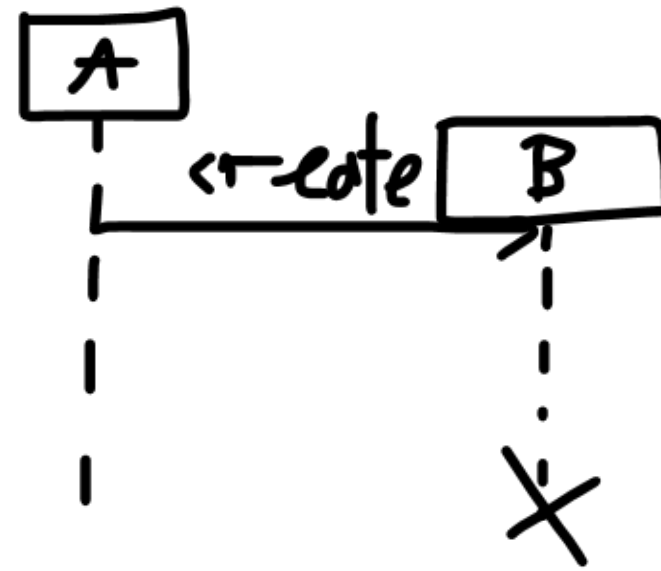
- **Symbol:** Gestrichelte Linie mit offener oder geschlossener Pfeilspitze.
- **Bedeutung:** Die Rückgabe der Kontrolle oder eines Wertes an den ursprünglichen Aufrufer

- **Selbstaufruf (Self Message)**

- **Symbol:** Ein Pfeil, der die Lebenslinie verlässt und wieder auf dieselbe zurückkehrt.
- **Bedeutung:** Ein Objekt ruft eine eigene Methode auf (interne Verarbeitung)

Sequenzdiagramm (Nachrichten/Messages)

- **Erzeugung & Zerstörung (Creation & Destruction)**
 - **Erzeugung (Create):** Ein Pfeil mit der Beschriftung «create», der auf den Kopf einer *neuen* Lebenslinie zeigt (diese beginnt erst an dieser vertikalen Position)
 - **Zerstörung (Destroy):** Ein großes **X** am Ende der Lebenslinie. Das Objekt wird aus dem Speicher entfernt

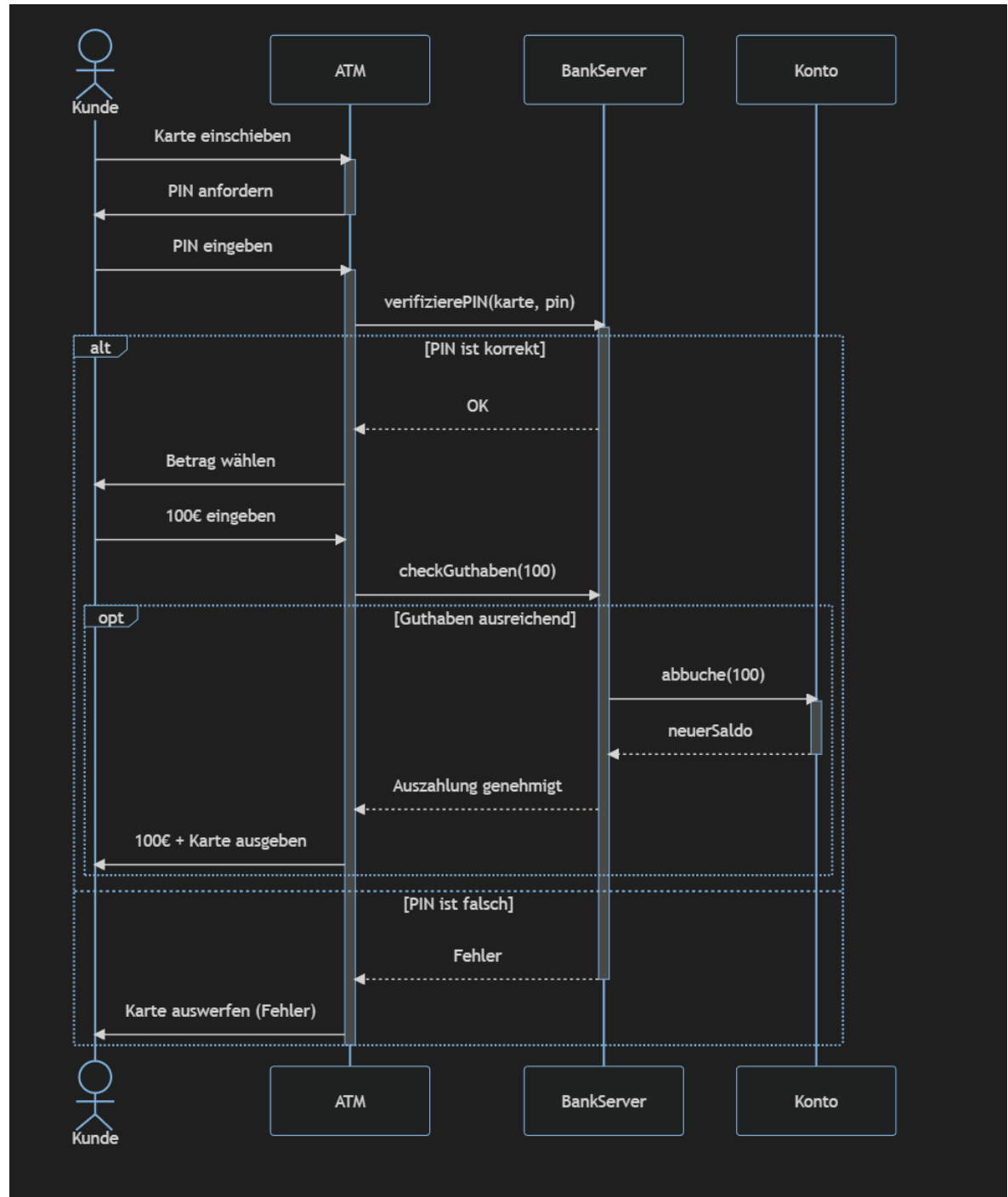


Strukturierungselemente (Interaktionsfragmente)

- Um komplexe Logik wie Schleifen oder Bedingungen darzustellen, werden **Kombinierte Fragmente** (Rahmen) verwendet.
- **Alternative (alt)**
 - **Funktion:** Entspricht einer if / else-Anweisung.
 - **Syntax:** Ein Rahmen mit dem Label alt.
 - gestrichelte horizontale Linie teilt den Rahmen in Segmente. Nur *eines* der Segmente wird ausgeführt, abhängig von der Bedingung (Guard), die in eckigen Klammern steht [Bedingung]
- **Option (opt):**
 - **Funktion:** Entspricht einer if-Anweisung ohne else
 - **Syntax:** Ein Rahmen mit dem Label opt. Der Inhalt wird nur ausgeführt, wenn die Bedingung [Guard] wahr ist

Strukturierungselemente (Interaktionsfragmente)

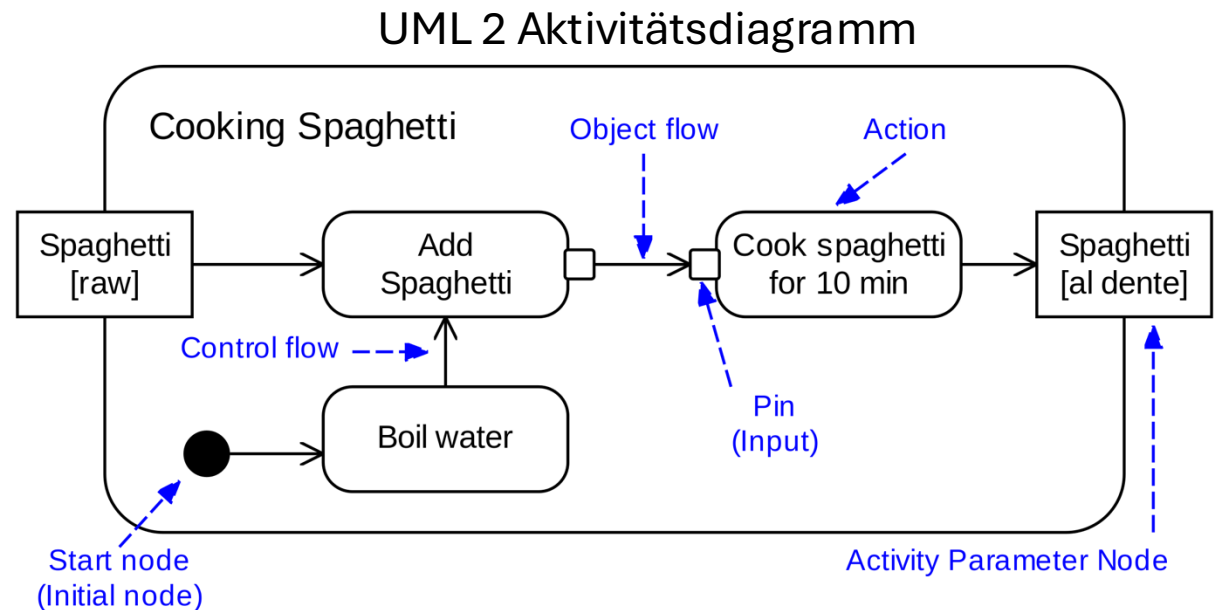
- **Schleife (loop)**
 - **Funktion:** Entspricht einer for oder while-Schleife
 - **Syntax:** Ein Rahmen mit dem Label loop. Oft steht daneben die Bedingung, z. B. loop [1..n] oder loop [solange Fehler]
- **Parallelität (par)**
 - **Funktion:** Gleichzeitige Ausführung (Multithreading)
 - **Syntax:** Ein Rahmen mit dem Label par. Die Aktionen in den verschiedenen Segmenten laufen zeitgleich ab
- **Referenz (ref):**
 - **Funktion:** Verweis auf ein anderes Sequenzdiagramm (Modularisierung)
 - **Syntax:** Ein Rahmen mit dem Label ref in der Mitte, der den Namen des anderen Diagramms enthält. Dient dazu, Details auszulagern



Aktivitätsdiagramm (Verhalten / Ablauf)

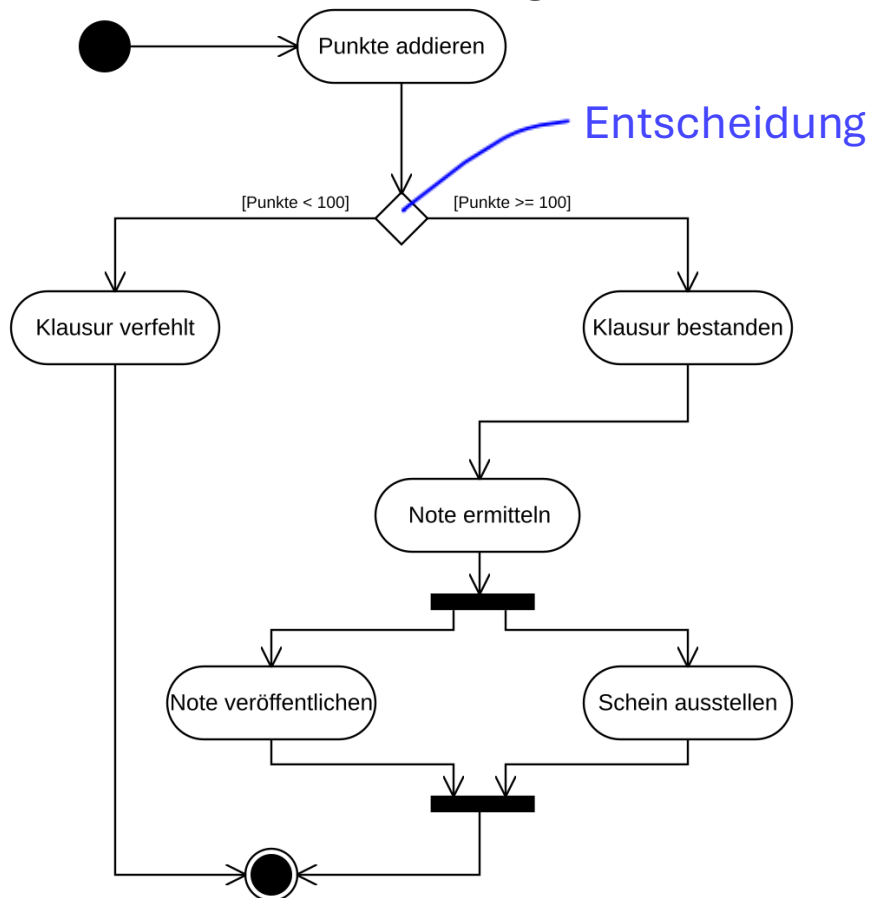
- **Zweck:**

Zeigt den flussorientierten Ablauf eines Prozesses (ähnlich einem Flussdiagramm). Es visualisiert Entscheidungen (If/Else), Schleifen und parallele Abläufe.

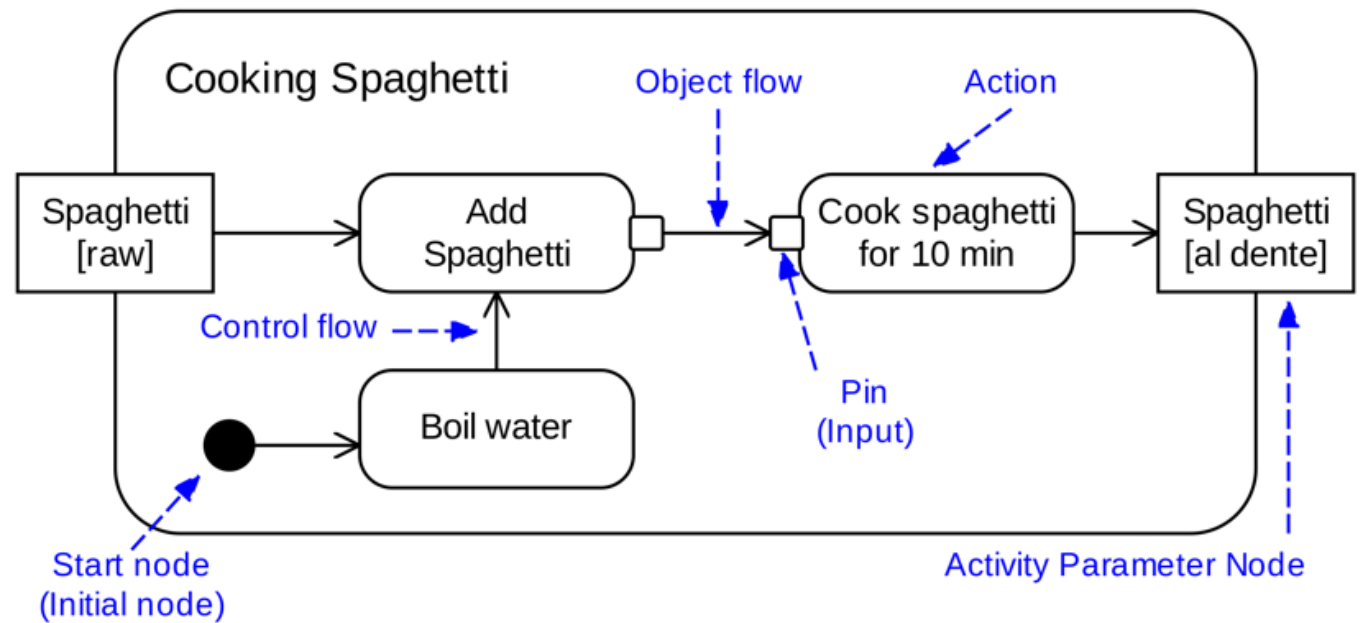


UML 1 vs UML 2 Aktivitätsdiagramm

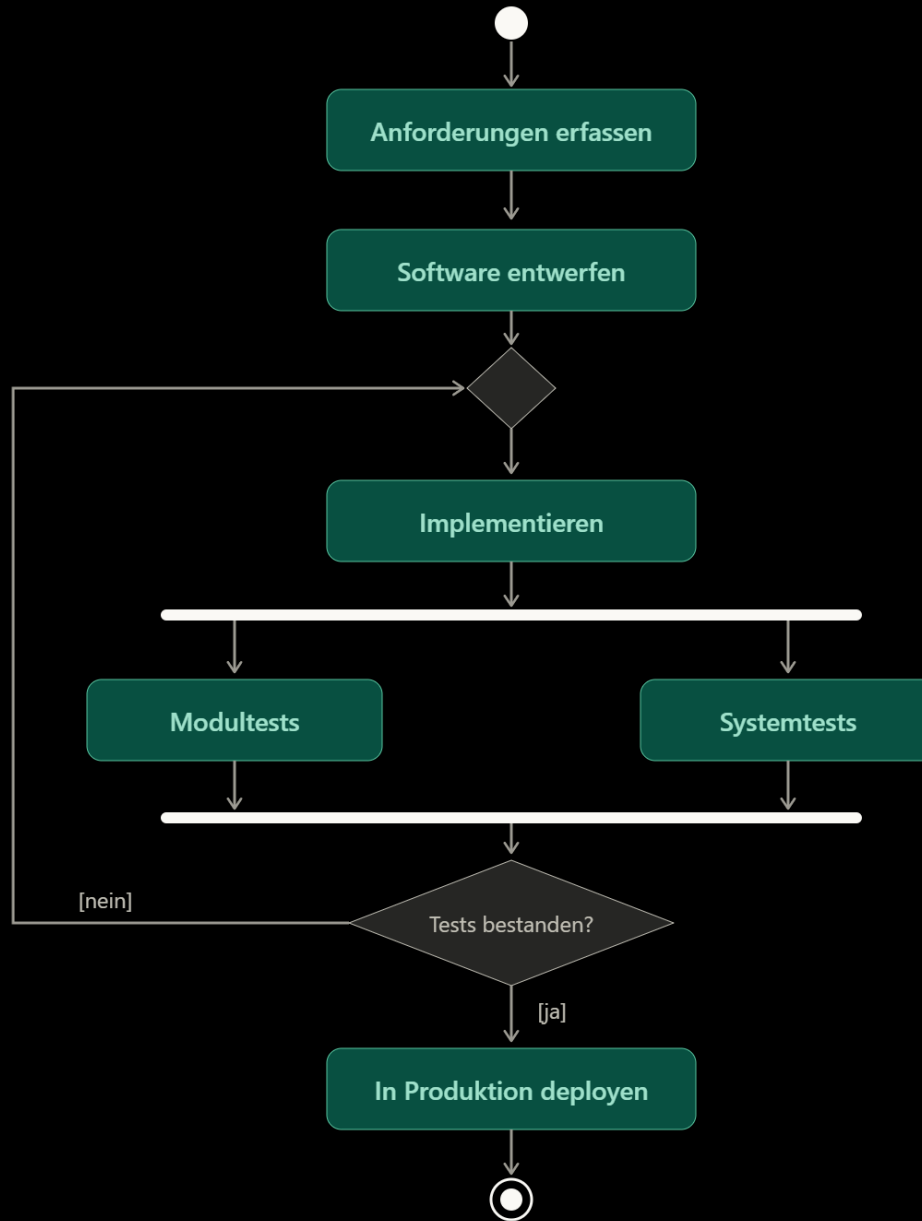
UML 1 Aktivitätsdiagramm



UML 2 Aktivitätsdiagramm



<https://de.wikipedia.org/wiki/Aktivit%C3%A4tsdiagramm#/media/Datei:Uml-Activity-Beispiel2.svg> CC BY-SA 3.0 Stern



Symbole

Startknoten

Endknoten:

Beendet alle
Aktivitäten

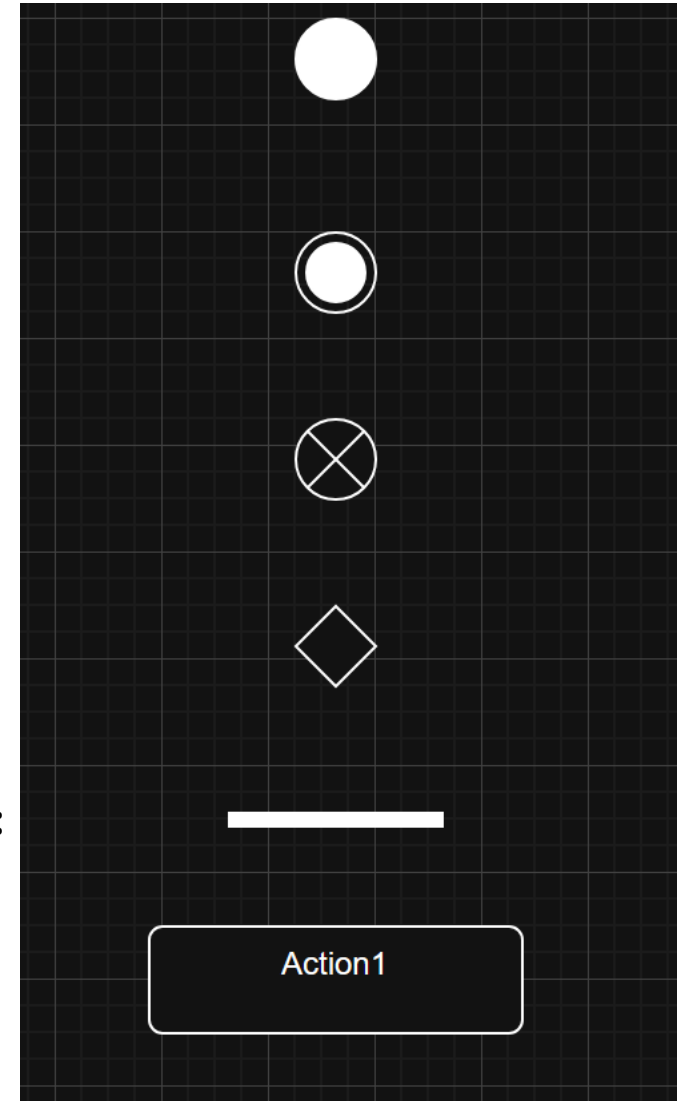
Ablaufende:

Beendet
aktuellen Fluss

**Verzweigung/
Zusammenführung**

Zusammenführung/Gabelung:
z.B. für parallele Verarbeitung

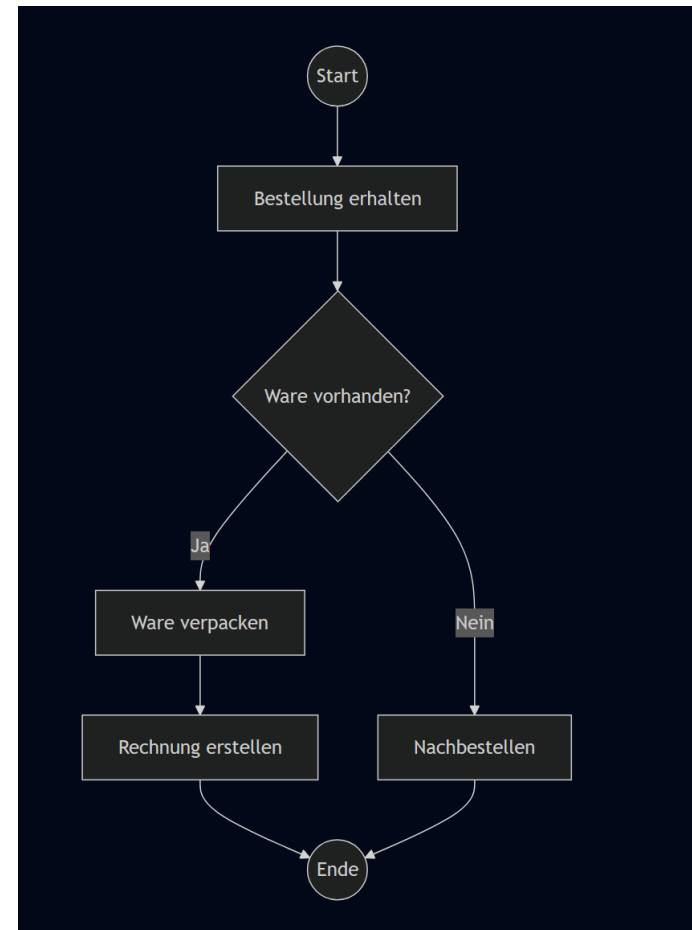
Aktivitäten (Schritte)



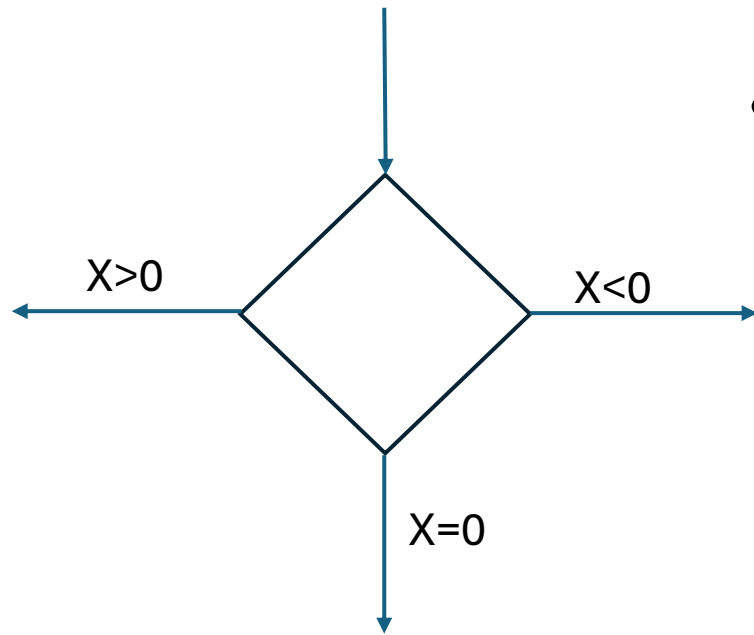
Aktivitätsdiagramm

```
def bestellprozess(ware_vorhanden):  
    print("Bestellung erhalten")  
  
    if ware_vorhanden:  
        print("Ware verpacken")  
        print("Rechnung erstellen")  
    else:  
        print("Nachbestellen")  
  
    print("Vorgang beendet")  
  
bestellprozess(True)
```

UML 1



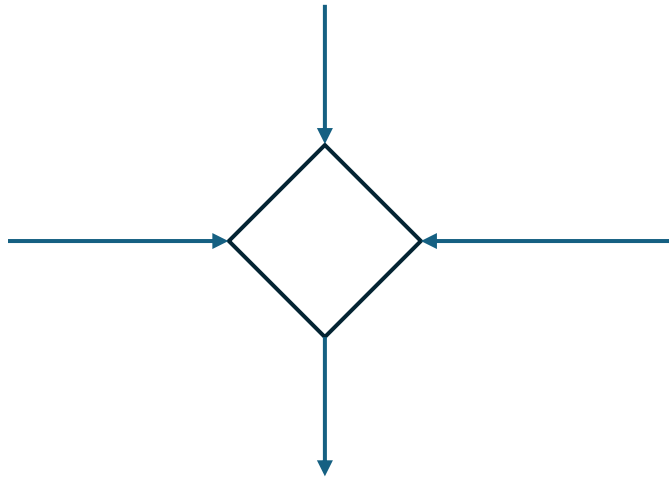
Aktivitätsdiagramm - Entscheidungen



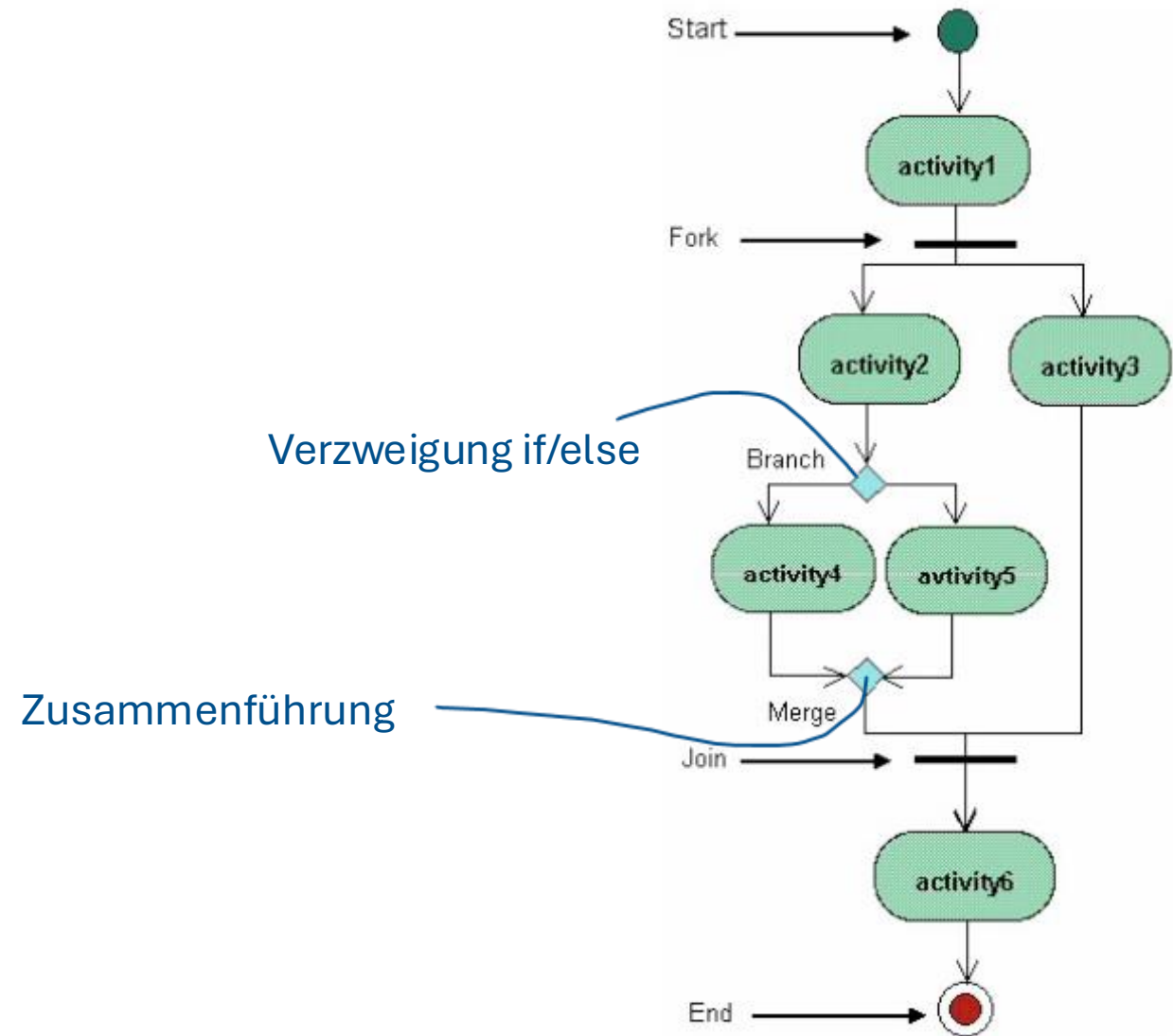
- Entscheidung/Verzweigung if/else

Aktivitätsdiagramm - Zusammenführungen

- **Zusammenführung/Merge (ODER):**
 - Wenn mindestens ein Pfeil „reinkommt“, dann geht es weiter
 - Verhält sich wie ein logisches ODER



Aktivitätsdiagramm - Verzweigungen

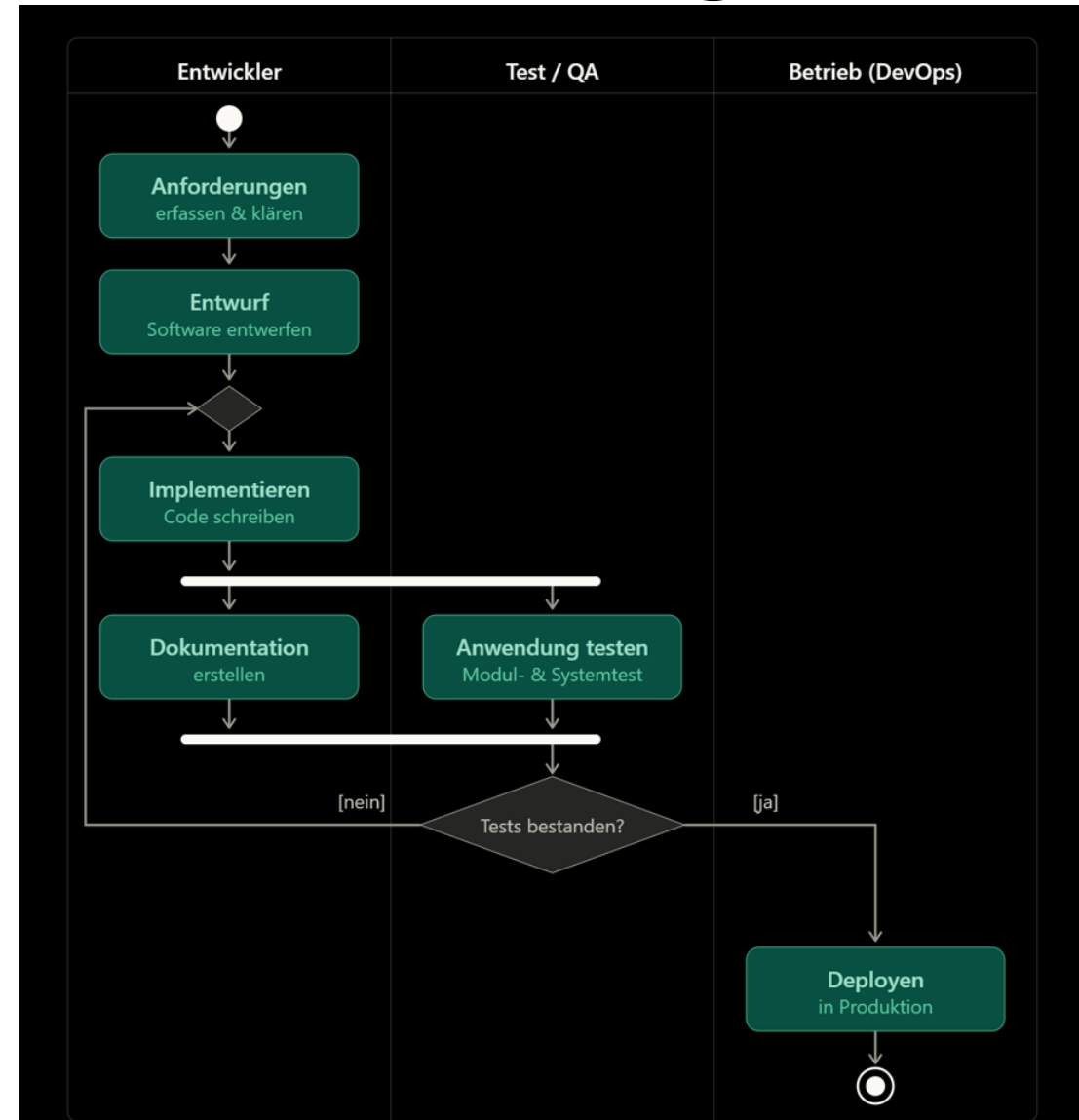


Swimlanes für Akteure - Aktivitätsdiagramm

Swimlanes (auch Partitions) strukturieren Aktivitätsdiagramme.

Diagramm wird in **vertikale oder horizontale Bereiche** unterteilt

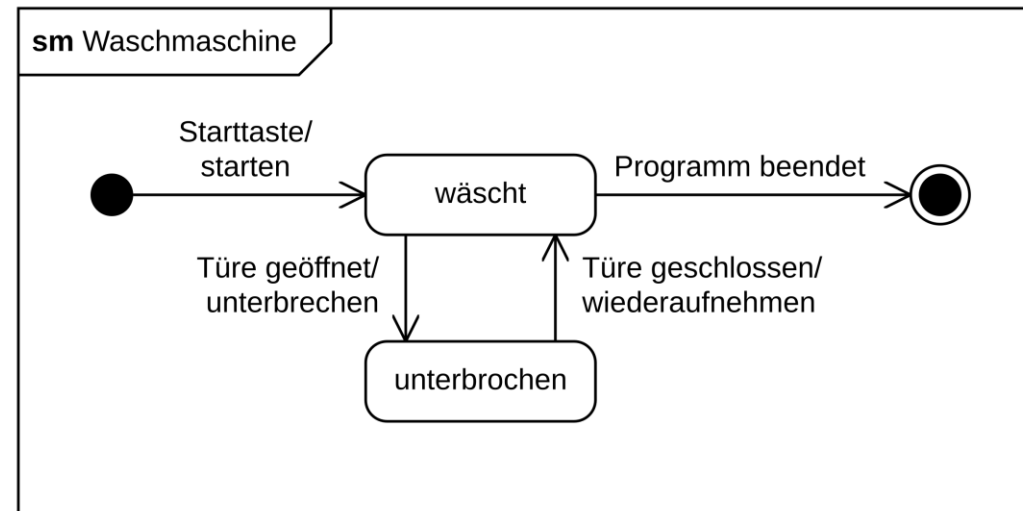
Jede Swimlane steht für:
eine Rolle (z. B. *Kunde*,
Sachbearbeiter)
ein System (z. B.
Webshop, *Thread1,2,3*
usw.)



Zustandsdiagramm (State Diagram)

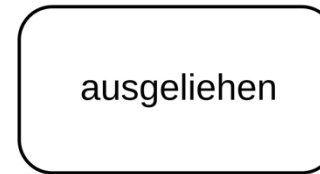
- **Zweck:**

Beschreibt die verschiedenen Zustände eines *einzelnen* Objekts und die Übergänge (Transitionen) durch Ereignisse. Sehr wichtig für Maschinensteuerungen oder den Lebenszyklus von Dokumenten.

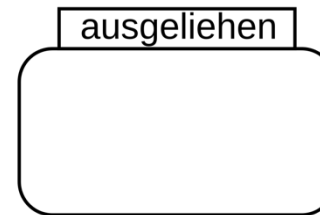


Zustände

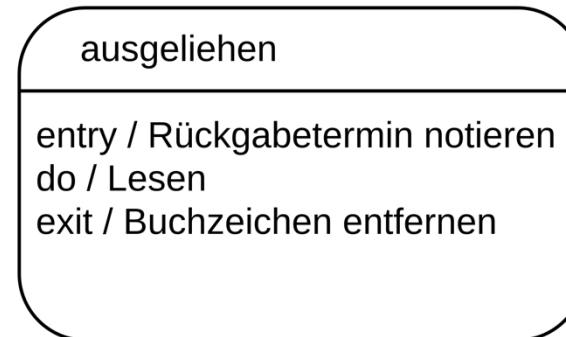
Ein Zustand wird im Diagramm als Rechteck mit abgerundeten Ecken dargestellt und mit dem Namen des Zustands beschriftet.



ein einfacher Zustand,
Standardnotation



alternative Notation



Zustand mit zugeordneten
Verhaltensspezifikationen

Zustandsdiagramm

Pseudo-Zustände

Pseudozustände (auch *Pseudostates*) sind spezielle Elemente in **UML-Zustandsdiagrammen**, die **keine echten Zustände** darstellen.

Sie beschreiben vielmehr **Steuerungs-, Start- oder Entscheidungslogik** innerhalb des Ablaufs einer Zustandsmaschine. In ihnen verweilt ein Objekt **nicht** – sie werden sofort „durchlaufen“.

●	Startzustand
×	Endzustand
	Vereinigung oder Gabelung
◇	Kreuzung oder Entscheidung
○	Eintrittspunkt
⊗	Austrittspunkt
⊙	flache Historie
⊙ [*]	tiefe Historie

History-Zustände in UML-Zustandsdiagrammen

- **Zweck**

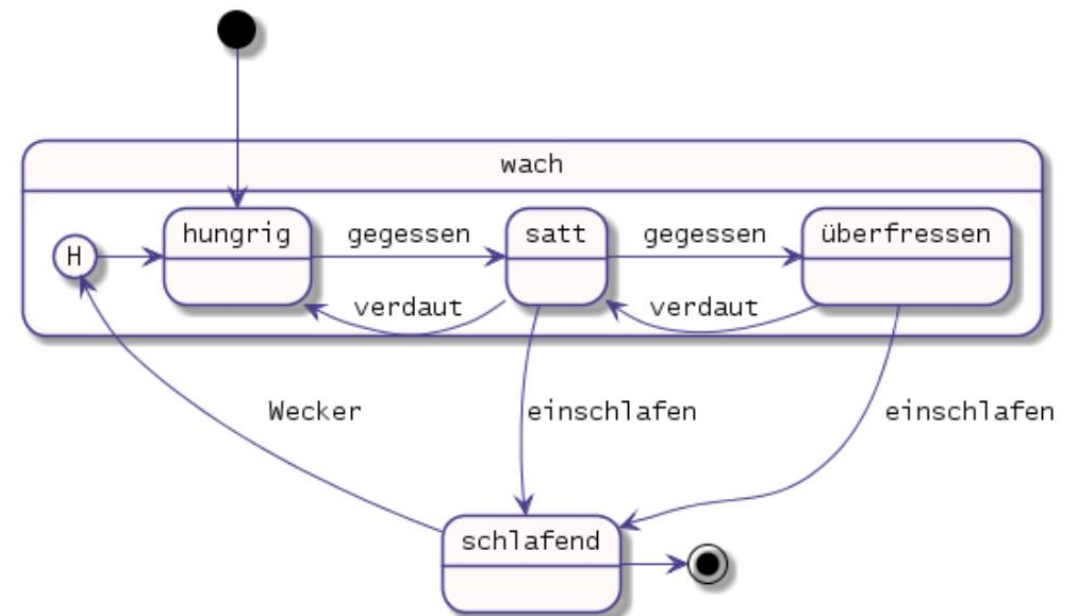
- Speichern den zuletzt aktiven Subzustand eines komplexen Zustands.
- Ermöglichen die Rückkehr zu genau diesem Subzustand nach einer äußeren Transition.

- **Funktionsweise**

- Komplexer Zustand wird verlassen.
- Aktiver Subzustand wird gespeichert.
- Rückkehr über den History-Zustand.
- Gespeicherter Subzustand wird reaktiviert.
- Alle **Entry-Aktivitäten** werden von außen nach innen ausgeführt.

Flacher History-Zustand (H)

- Stellt den **zuletzt aktiven Subzustand auf derselben Ebene** wieder her.
- Berücksichtigt keine tiefer verschachtelten Zustände.
- Wenn man vom Zustand schlafend per Wecker geweckt wird, nimmt man wieder den Subzustand von wach ein, den man vor dem Einschlafen hatte (also **satt** oder **überfressen**)
- Wenn der Zustand bislang nicht betreten wurde, wird der Zustand auf den **H** zeigt angenommen (**hungrig**)

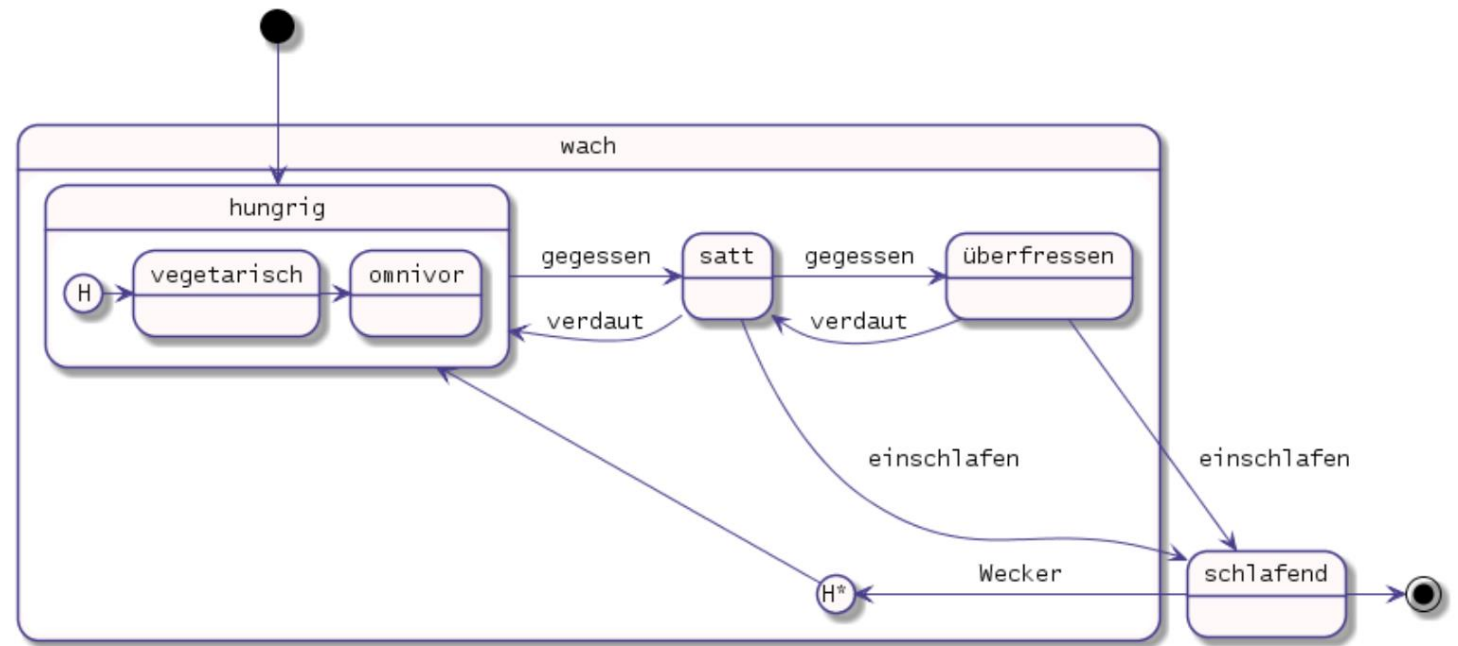


CC BY 4.0 Hannes Stein
<https://oer-informatik.gislab.io/uml/umlzustand/uml-zustandsdiagramm-plantuml.html>

Flache Historie eines zusammengesetzten Zustands

Tiefer History-Zustand (H*)

- Stellt den **zuletzt aktiven Subzustand über alle Schachtelungsebenen hinweg** wieder her.
- Berücksichtigt die gesamte Zustands-Hierarchie.
- Wenn der Wecker klingelt und der letzte Subzustand von wach war hungrig, dann wechselt das System auch unmittelbar wieder in den vorigen Subzustand von hungrig (also **vegetarisch** oder **omnivor**)

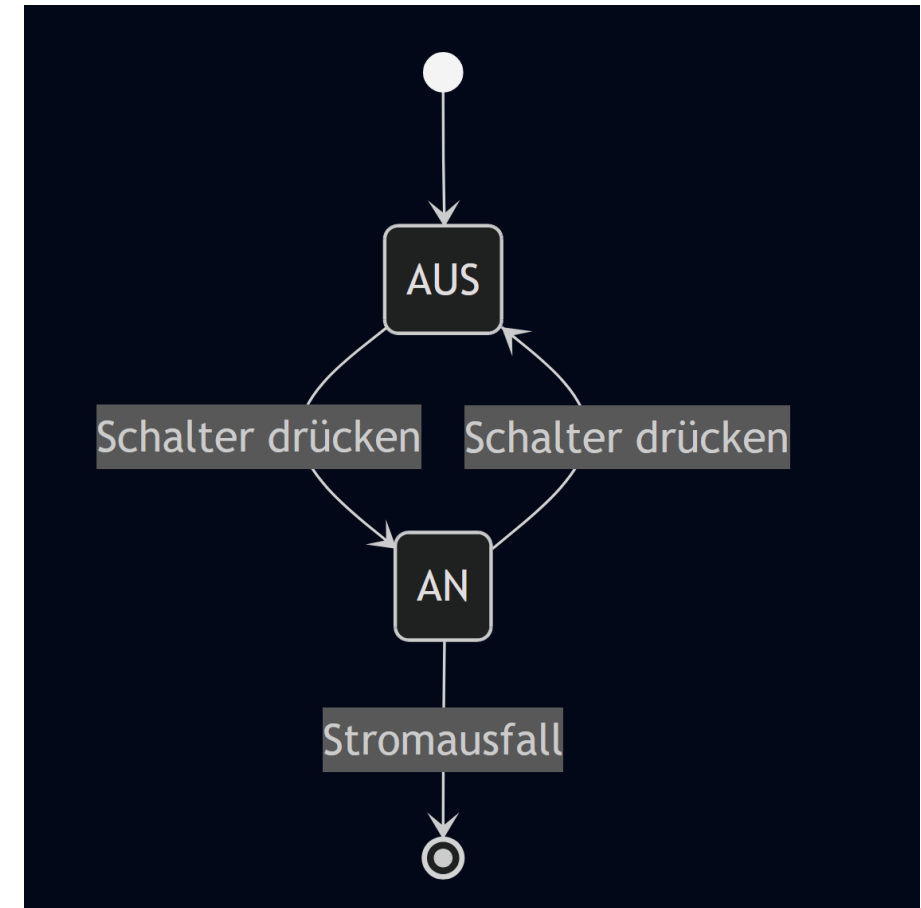


CC BY 4.0 Hannes Stein
<https://oer-informatik.github.io/unl/unl-zustand/unl-zustandsdiagramm-plansunl.html>

tiefe Historie

Zustandsdiagramm

```
class Lampe:  
    def __init__(self):  
        self.state = "AUS"  
  
    def schalter_druecken(self):  
        if self.state == "AUS":  
            self.state = "AN"  
            print("Licht geht an")  
        else:  
            self.state = "AUS"  
            print("Licht geht aus")  
  
l = Lampe()  
l.schalter_druecken() # Zustand wird AN
```

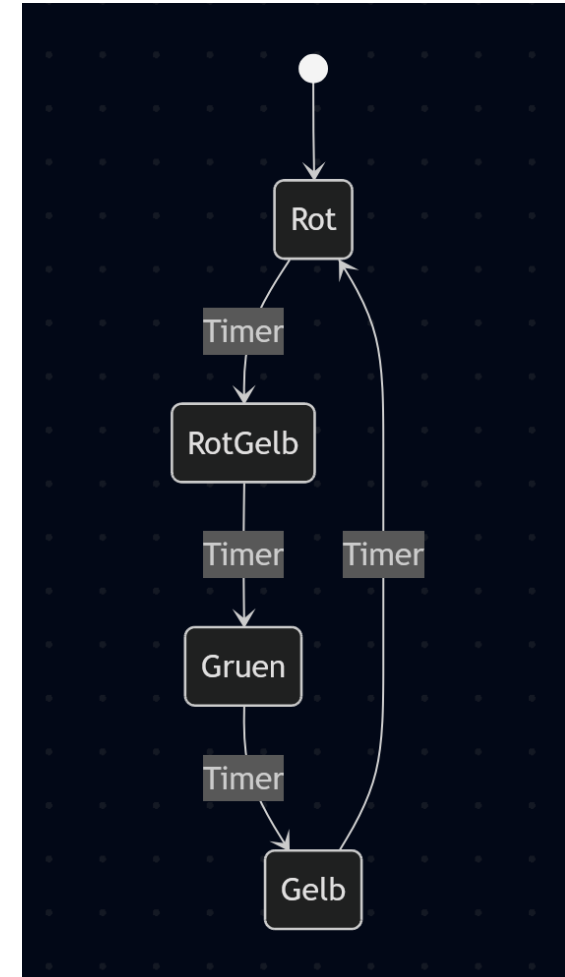


Aufgabe

- **Szenario:** Eine **Ampel**.
- Welches Diagramm eignet sich am besten, um die Phasen (Rot, Gelb, Grün) und deren Wechsel zu modellieren?
- Erstelle ein geeignetes UML Diagramm

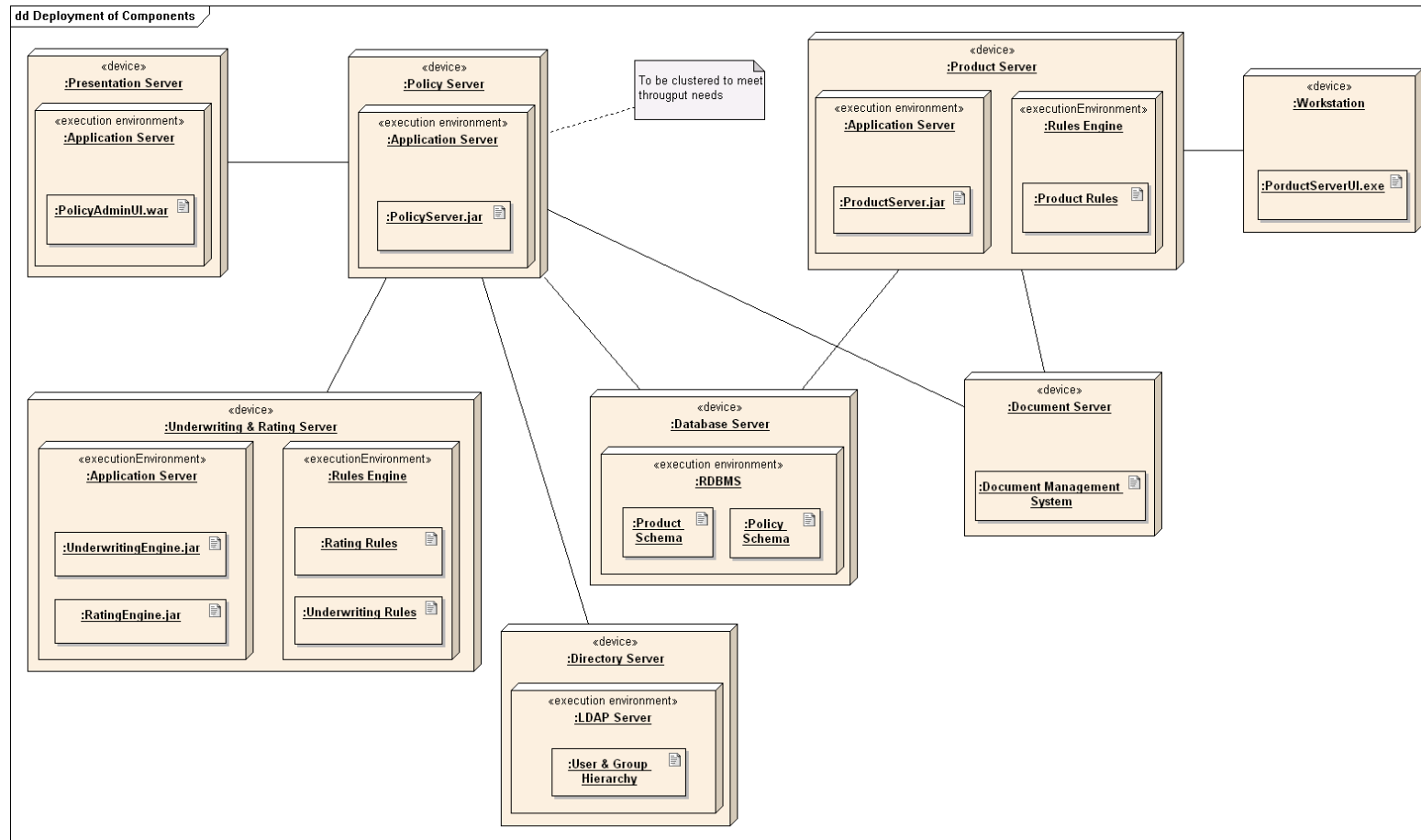
Lösung

- **1. Antwort:** Das **Zustandsdiagramm** (State Diagram), da es den Lebenszyklus und die Statusänderungen eines Objekts beschreibt.

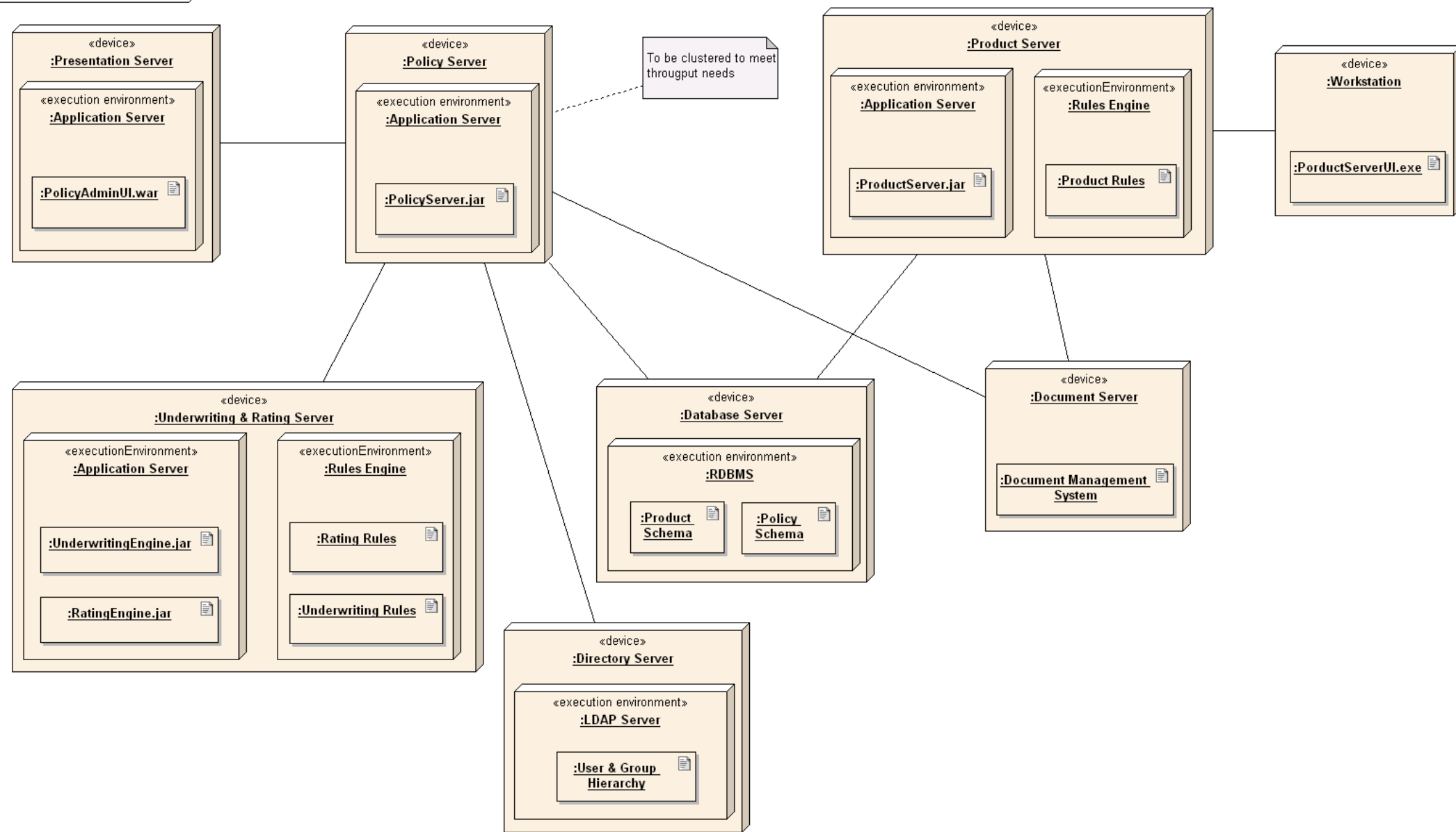


Verteilungsdiagramm (Deployment Diagramm)

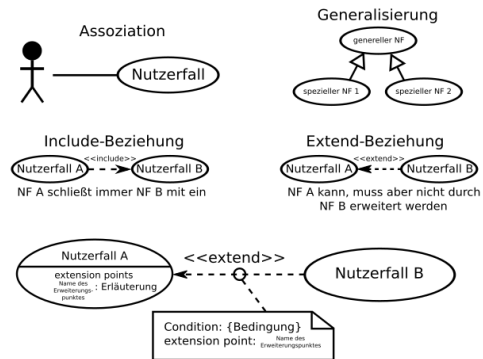
- Es gibt zwei Arten von Knoten:
- **Geräteknoten (Device Node)**
- **Ausführungsumgebungs-Knoten (Execution Environment Node)**
- **Geräteknoten** sind **physische Rechenressourcen** mit Rechenleistung, Speicher und Diensten zur Ausführung von Software
- Ein **Ausführungsumgebungs-Knoten (EEN)** ist eine **softwarebasierte Rechenressource**, die innerhalb eines übergeordneten Knotens läuft und selbst einen Dienst bereitstellt, um andere ausführbare Softwareelemente zu hosten und auszuführen.



dd Deployment of Components



Nutzerfalldiagramm



Zustandsdiagramm

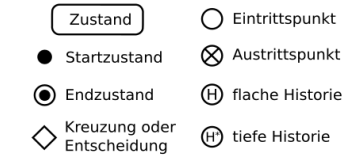
Verhaltenszustandsmaschine (BSM)

Transition ::= [Auslöser] [[Bedingung]] / [[Aktion]]
Auslöser ::= Ereignisse ((Zuweisungen))
Ereignisse ::= Ereignisse ((Ereignisse))*
Zuweisungen ::= Zuweisung ((Zuweisung))*
Zuweisung ::= Attribut | Attribut : Typ

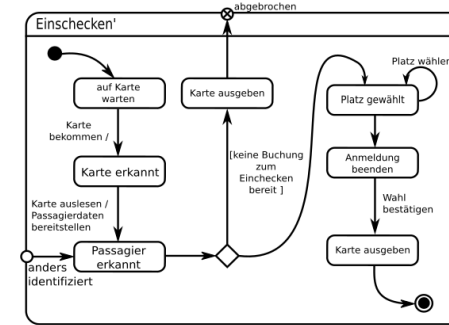
Protokollzustandsmaschine (PSM)

Transition ::= [[Vorbedingung]] Methodenaufruf / [[Nachbedingung]]

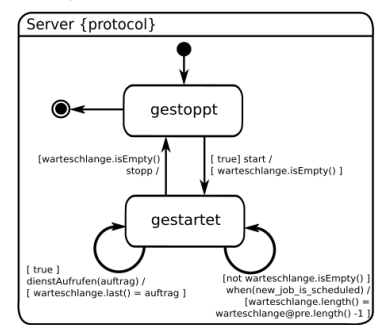
Notationen:



Beispiel: Verhaltenszustandsmaschine



Beispiel: Protokollzustandsmaschine



Klassendiagramm

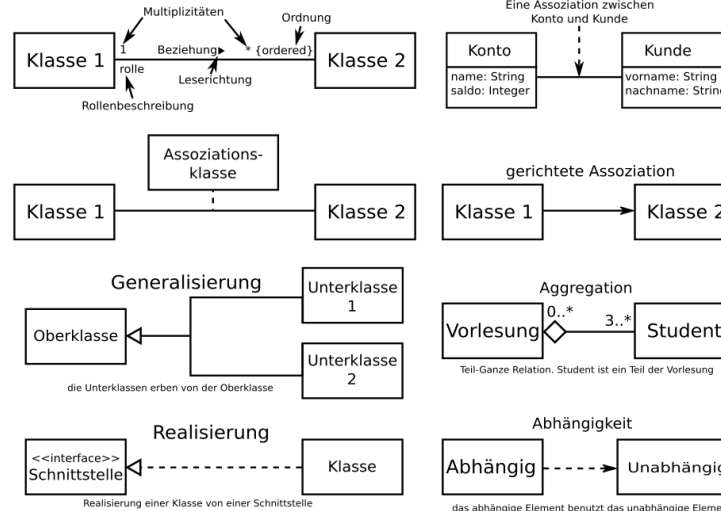
Analysemodell (fachliche Sicht)

- Klassen sind Fachbegriffe
- Attribute
 - i. Allg. ohne Datentypen
 - ggfs. mit Multiplizitäten
- Methoden ohne Parameter und Rückgabewert
- Bidirektionale Assoziationen/Aggregationen
- Beschreibung von Assoziationen (Multiplizitäten, Rollennamen, Assoziationsnamen (mit Leserichtung))
 - ggfs. mit Qualifier
- Assoziationsklassen und n-äre Beziehungen
- Generalisierung / Spezialisierung (Vererbung)
- Abgeleitete Attribute und Methoden
- Aufzählungen (Enumerationen)

Entwurfsmodell (fachliche+tech. Sicht)

- Klassen -> abstrakt, Interface, Stereotyp, ggf. Klassen streichen, hinzufügen, umbenennen
- Attribute -> Sichtbarkeiten, Ableitung, Klassenattribute, Initialisierung, weitere spezielle Eigenschaften
- Operationen -> Parameter, Sichtbarkeiten, Rückgabewert, Klassenoperation
- Assoziationen -> gerichtet, geordnet / sortiert
- Auflösen von Assoziationsklassen / n-äre Beziehungen
- Abhängigkeiten
- Packages
- Hilfsmethoden (Konstruktoren, getter/setter, toString() u.a.)

Eine Assoziation beschreibt eine Beziehung zwischen zwei oder mehr Klassen. An den Enden von Assoziationen sind häufig Multiplizitäten vermerkt. Diese drücken aus, wie viele dieser Objekte in Relation zu den anderen Objekten dieser Assoziation stehen.

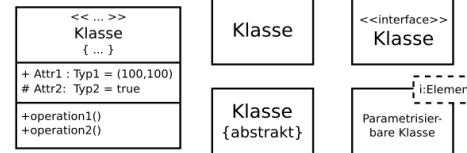


Syntax für Attribute

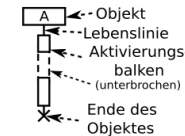
Sichtbarkeit Attributname : Paket :: Typ [Multiplizität Ordnung] = Initialwert {Eigenschaftswerte}
Eigenschaftswerte : {readOnly}, {ordered}, {composite}

Syntax für Operationen

Sichtbarkeit Operationsname (Parameterliste) : Rückgabety
Sichtbarkeit:
+ public element
protected element
- private element
~ package element
Parameterliste: Richtung Name : Typ = Standardwert
Richtung: in, out, inout



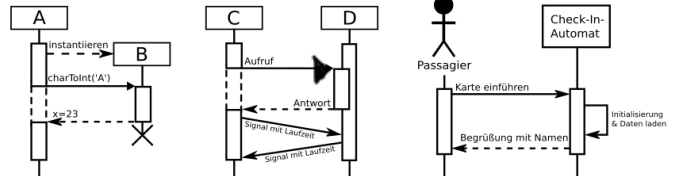
Sequenzdiagramm



Beschreibung:
Ein synchroner Aufruf unterbricht den Aktivierungsbalken solange, bis eine synchrone Antwort eintrifft. Eine asynchrone Nachricht verändert nichts am Aktivierungsbalken.

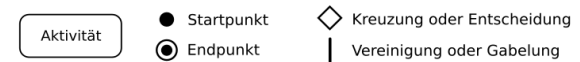
Notationen:

Nachricht ::= Aufruf | Antwort | Signal
→ Aufruf (synchrone Nachricht)
← Antwort (synchrone Nachricht)
→ Signal (asynchrone Nachricht) (schräg bei relevanter Laufzeit)



Aktivitätsdiagramm

Notationen:



Beispiel: Aktivitätsdiagramm

